

# Weekly 엔터프라이즈 중장기 기술 로드맵 (Product Roadmap Plan)

문서 버전: v0.25.0 (현재) ~ v1.0-  
VISION

작성일자: 2026년 8월 9일

최종 정렬: 2026년 8월 21일  
(v0.25.0 기준)

문서 분류: 비즈니스 및 아키텍처 중장  
기 로드맵 (Strategic Product  
Roadmap)

## 1. 현재 위치와 다음 방향

v0.10.0까지 "주간보고를 잘 만드는 시스템" 은 상당 부분 완성되었습니다. 작성·승인, 기간 집계, 경영 인사이드, PPTX 내보내기, Confluence 근거 기반 초안, 내용 검색, 관리자 분석이 모두 동작합니다.

다음 단계의 성격은 다릅니다. "회사가 지금 무슨 일을 하고 있고, 무엇이 바뀌었고, 무엇이 막혀 있고, 왜 그렇게 결정했는지 아는 시스템" 으로 전환합니다. 이를 위한 핵심 축은 네 개입니다.

=====

[Weekly 다음 단계 4개 핵심 축]

=====

1. WorkItem

주차를 초월하는 업무 식별자. 나머지 세 축의 전제 조건

2. 주간 변화 요약

지난주 대비 신규·진척·완료·지연·재개 자동 판별

3. Decision & Ask

결정사항과 상위 조직 요청을 이슈와 분리해 기록

4. Dependency

업무 간 선행·차단 관계로 부서 간 병목 조기 발견

=====

### 1.1 왜 WorkItem이 먼저인가

현재 데이터 모델은 Report → ReportItem 이며, 같은 업무가 매주 새 행으로 다시 생성됩니다. 주차를 넘는 업무 동일성은 기간 집계에서 제목 정규화와 토큰 포함 관계로 추정하고 있을 뿐, 저장된 사실이 아닙니다.

변화 요약, Aging, Dependency, Risk 예측은 모두 "이 업무가 지난주의 그 업무와 같은가"에 답할 수 있어야 성립합니다. WorkItem 없이 각 기능이 제각각 추정 로직을 다시 구현하면 기능마다 판정이 어긋납니다.

## 2. 완료된 마일스톤 (v0.1.0 ~ v0.25.0)

```
=====
[v0.1.0] Single Container, PPTX Zip Engine, Keycloak OIDC, RBAC, MCP 1.0
[v0.2.0] 자유 텍스트 AI 초안, 과거 PPTX Import, 사내 AI Gateway
[v0.3.0] Confluence 6.9.1 증분 수집과 사용자별 자동 초안
[v0.4.0] 주간보고 생애주기와 연동 개선
[v0.5.0] 보고서 복제, 업그레이드 안전 비밀 설정(WEEKLY_ENCRYPTION_KEY)
[v0.6.0] 기간 업무보고(월·분기·반기·연간), 중복 제거, 경영 인사이트, 업무 시각화
[v0.7.0] AI 원자 사실 구조화, Confluence 근거 검증, PPTX Import 구조 보존
[v0.7.1] 보고서 수정 장애 복구, 문자 수 기준 입력 검증
[v0.7.2] 비밀 설정 유실 방지 기동 차단, PPTX 드래그 앤 드롭
[v0.8.0] 화면 캡처 슬라이드, 기간 보고 전용 분면 레이아웃
[v0.9.0] 빠른 이동 팔레트, 보고서 내용 검색, 화면 상태 딥링크
[v0.9.1] 가져오기 검토 화면 반응형 정리
[v0.10.0] 관리자 분석: 한국어 키워드 추출, 워드 클라우드, 조직·참여 분석
[v0.11.0] WorkItem 도입, 업무 Aging 추적, 상위 조직 요청 필드
[v0.12.0] 3단계 검색(정확·표기 유사·의미 유사), 표기 변형 합산 워드 클라우드
[v0.13.0] 회의 모드와 발표 모드, 경영 요약, 조직 간 중복·협업 분석, 인수인계
[v0.14.0] PPTX 내보내기가 있는 모든 화면의 발표 모드
[v0.14.1] 발표 모드 화면 캡처 슬라이드
[v0.14.2] 첨부 파일 유실 진단, 발표 모드 이미지 정렬 수정
[v0.15.0] 지난주 계획 이어받기, 임베딩 신선도 판정, 백업·복구 도구
[v0.16.0] 편집 내용 유실 방지, 업무 병합·분리, 로그인 시도 제한
[v0.17.0] 주간 변화 요약, 보고 품질 점검, 재개 업무 판정 누락 수정
[v0.18.0] 보고서 조회 색인, 제출 마감 설정과 시간대 오펜 수정, 기간 보고의 업무 식별 일원화
[v0.19.0] CSV 수식 주입 차단, 세션 만료 복구, PPTX 내보내기 메모리 축소
[v0.20.0] 발표 모드 자동 맞춤, 발표자 화면, 슬라이드 목록·번호 이동
[v0.21.0] 발표 테마 3종, 회사 템플릿 강조색 연동, 읽을 수 있는 최소 글자 크기
[v0.22.0] 기동 차단 해소와 재개 가능한 백필, 휴대폰 메뉴 접근성
[v0.23.0] 주차 격자 변경 안전장치, 조회 가능한 감사 이력, 오류 추적 ID 노출
[v0.24.0] 작성 화면 우선순위 정리, 저장 상태 표시, 대화상자 동작 통일
[v0.25.0] 업무 인사이트 응답 상한과 순위, 제목 토큰화 재사용
=====
```

기존 로드맵의 다음 항목은 계속 유효하며 이 문서의 후반 단계에 배치합니다.

- **다중 PPTX 템플릿:** 사업부·부서별 서로 다른 양식 동적 선택
- **미제출자 자동 알림:** Slack·Teams·사내 메일 (v0.10.0의 미제출 현황 데이터를 그대로 활용)
- **Source-to-Report Agent:** 커밋·결재·Jira 근거 수집 후 후보 생성

## 3. 버전별 계획

### 3.1 v0.11 — WorkItem과 업무 Aging (완료)

- **WorkItem** 도입: `id`, `title`, `owner`, `organization`, `status`, `startDate`, `dueDate`, `progress`, `parentWorkItemId`. `ReportItem` 은 `workItemId` 로 연결되어 주차별 스냅샷 역할만 맡습니다.
- **과거 데이터 시딩**: 기간 집계의 제목 정규화·토큰 포함 병합 로직을 재사용해 기존 보고서에서 WorkItem 후보를 생성하고, 사용자가 확인·분리·병합합니다. 자동 판정을 확정으로 쓰지 않습니다.
- **업무 Aging**: WorkItem 단위로 지속 주차, 이월 횟수, 동일 이슈 지속 기간, 진척 정체 구간을 추적합니다. 현재 기간 집계가 계산하는 값을 업무 생애주기로 옮깁니다.
- **Management Ask**: 이슈와 분리된 `상위 조직 요청` 필드를 함께 도입합니다. 난이도가 가장 낮고 임원 보고 가치가 가장 큰 항목이므로 WorkItem과 같은 릴리즈에 실습니다.

### 3.2 v0.12 — 검색 품질과 선택 확장 활용 (완료)

로드맵 후반의 의미 기반 검색을 앞당겨 기반만 먼저 놓았습니다. 4.3의 선행 결정이 실제 운영 환경에서 확인 가능해졌기 때문입니다.

- **3단계 검색**: 정확 일치 → `pg_trgm` 표기 유사 → `pgvector` 의미 유사. 앞 단계 결과가 5건 미만일 때만 다음 단계를 수행하고, 각 단계는 앞 단계가 찾은 보고서를 건너뜁니다.
- **확장은 선택**: 두 확장이 모두 없어도 정확 일치 검색이 그대로 동작합니다. 마이그레이션은 확장이 없으면 활성화를 건너뛰고 진행합니다.
- **표기 변형 합산 워드 클라우드**: 띄어쓰기만 다른 표기를 하나로 합산합니다. 유사도가 아니라 공백 제거 후 정확 일치로 판정합니다(4.3의 오판 금지 원칙 적용).
- **임계값은 설정으로**: 코사인 점수 범위는 임베딩 모델마다 다르므로 고정값으로 둘 수 없습니다. 관리자 화면의 임베딩 현황과 함께 조정합니다.

### 3.3 v0.13 — 회의·경영 관점과 조직 간 업무 관계 (완료)

WorkItem이 자리 잡으면서 로드맵 후반의 Meeting Mode(구 3.6)와 Executive Digest(구 3.9)를 앞당겨 구현했습니다. 두 기능 모두 WorkItem 스냅샷에서 결정적으로 파생되므로 선행 조건이 이미 충족돼 있었습니다.

- **회의 모드와 발표 모드**: 결정 필요·신규 이슈·지속 이슈·변경점·보고 누락만 남기고, 전체 화면 슬라이드로 회의에서 바로 진행합니다.

- **경영 요약:** 관측 사실의 가중 합으로 최대 10건을 선정하고 선정 근거를 항상 함께 제시합니다(4.3 원칙).
- **업무 인사이트:** 조직 간 중복 의심, 유사 업무, 협업 지도, 반복 운영 업무. 모두 사람이 확인할 후보로 제시합니다.
- **인수인계:** 진행 경과, 이슈 해소 여부, 미해결 사항을 기록에서 모읍니다.
- **과거 사례 검색:** 자연어로 유사 업무와 이슈 해결 경과를 찾습니다. v0.12의 임베딩을 선택 가속 경로로 사용합니다.

### 3.4 v0.14 — 발표 모드 일반화 (완료)

발표 모드를 회의 모드 전용에서 분리해, **PPTX를 내보낼 수 있는 모든 화면**에서 사용할 수 있게 했습니다. 내보내기와 발표는 같은 내용을 다른 매체로 전달하는 것이므로 한쪽만 제공할 이유가 없습니다.

- 대상: 내 주간보고, 과거 보고, 팀 주간보고, 기간 업무보고, 회의 모드
- 발표 컴포넌트는 슬라이드 배열만 받으며 보고서·기간 집계·회의 안건을 알지 못합니다. 텍 구성은 각 화면이 담당합니다.
- 내 주간보고는 저장하지 않은 편집 내용도 그대로 발표합니다. 작성 직후 그대로 설명하는 경우가 많기 때문입니다.
- v0.14.1에서 첨부한 화면 캡처도 슬라이드로 포함합니다. 배치는 첨부 패널의 **본문 앞** · **본문 뒤** 표시를 따르며, PPTX와 달리 표지를 항상 먼저 보여 줍니다.

### 3.5 v0.15 — 작성 보조와 데이터 신뢰성 (완료)

v0.15.0은 이 단계의 원래 주제인 주간 변화 요약보다, 그 앞에 있던 세 가지를 먼저 처리했습니다. 두 가지는 조용히 틀린 답을 내고 있었고, 하나는 매주 모든 사용자가 치르는 비용이었습니다.

- **지난주 계획 이어받기 (완료):** 작성 화면이 직전 보고서의 차주 계획을 해당 업무 옆에 그대로 보여 주고, 아직 보고하지 않은 계획은 목록으로 모아 한 번에 업무 항목으로 옮깁니다. 짝짓기는 **WorkItem** 식별자 우선, 없으면 공백 제거 후 정확 일치입니다(4.5의 원칙 적용). 직전 \*주차\*가 아니라 직전 \*보고서\* 기준이므로 한 주를 건너뛰어도 이어집니다.
- **임베딩 신선도 판정 (완료):** v0.11에서 보고 항목을 행 갱신 방식으로 바꾼 뒤, 임베딩 대상 판정이 "임베딩 행이 없는 항목"이어서 수정된 내용이 영원히 재임베딩되지 않았습니다. 판정을 본문 해시 비교로 바꾸고, 관리자 화면에 갱신 대기 건수를 노출합니다. 해시는 PostgreSQL이 계산합니다. 판정이 SQL 조건이어야 하기 때문입니다.
- **백업·복구 도구 (완료):** 4.2의 두 저장소 문제를 문서가 아니라 도구로 다룹니다. `scripts/weekly-backup.sh`가 DB와 상태 볼륨을 같은 복구 지점으로 묶고, 참조된 첨부 파일이 실제로 보관됐는지 검증

합니다.

- 주간 변화 요약 (**v0.17.0**에서 완료): 아래 3.5.1을 보십시오.
- 보고 품질 점검 (**v0.17.0**에서 완료): 아래 3.5.1을 보십시오.
- **AI 표현 점검**(선택, 남음): 모호한 표현과 근거 없는 완료 주장은 AI Gateway가 켜져 있을 때만 추가로 제시합니다. 규칙 기반 점검은 오프라인에서도 항상 동작합니다.

### 3.5.1 v0.17 — 주간 변화 요약과 보고 품질 점검 (완료)

- 판정은 하나만 둡니다. 변화 분류를 회의 모드와 변화 요약이 **같은 함수**로 수행합니다. 1.1이 WorkItem 을 먼저 도입한 이유와 같습니다. 화면마다 비교를 다시 구현하면 같은 업무가 한 곳에서는 완료로, 다른 곳에서는 진척으로 보이고 읽는 사람이 둘 다 믿지 않게 됩니다.
- 재개 판정 누락 수정: v0.13의 회의 모드는 한 주 이상 쉬었다가 다시 보고된 업무를 **어느 구획에도 넣지 못하고 조용히 버리고 있었습니다**. 조용했다 돌아온 일은 회의가 가장 듣고 싶어 하는 소식입니다. **재개** 를 정식 분류로 넣어 해소했습니다.
- 그대로 진행 중은 세되 보여 주지 않습니다. 요약이 보고서에 이미 적힌 내용을 반복하면 요약이 아닙니다. 완료 후 보고에서 빠진 업무도 누락이 아니므로 제외합니다.
- 보고 품질 점검은 작성 중인 초안을 봅니다. 저장본만 볼 수 있는 점검은 작성자가 이미 지나간 버전에 대해 답합니다. 규칙 네 가지(진척도 역행, 계획 반복, 실적 누락, 이슈 장기화)는 결정적이며 AI가 꺼진 환경에서도 동작합니다. 이미 상위 조직 요청으로 올린 이슈에는 다르게 말합니다. 조치한 사람에게 조치하라고 반복하면 점검 전체가 무시됩니다.

### 3.5.2 v0.18 — 성능, 지표 정확성, 판정 일원화 (완료)

- 보고서 조회 색인: `report_items.report_id` 에 색인이 없어 보고서를 열 때마다 테이블 전체를 훑고 있었습니다. PostgreSQL은 외래 키의 참조하는 쪽을 색인하지 않습니다. 8만 행 기준 5.07ms·1,144버퍼에서 0.074ms·11버퍼로 줄었습니다. `report_comments` , `report_status_history` 도 같이 처리했습니다.
- 제출 마감 설정과 시간대 오판 수정: 정시 판정이 `submitted_at::date <= week_start + 7` 이었습니다. 조직 정책을 코드에 박아 둔 것이고, `::date` 가 데이터베이스 세션 시간대(UTC)로 계산돼 **마감 다음 날 오전 9시 이전 제출이 전부 정시로 집계**됐습니다. 마감을 `주차 시작일 + N일 + H시` 로 설정하게 하고 판정을 서비스 시간대로 명시했습니다. 화면은 기준을 함께 표시합니다.
- 기간 보고의 업무 식별 일원화: 기간 집계만 여전히 제목 유사도 80%로 같은 업무를 다시 추측하고 있습니다. 한 담당자 안에서는 v0.16.0의 병합·분리 결과를 그대로 쓰고 추측하지 않습니다. 담당자 사이에

서는 사람을 가로지르는 저장된 식별자가 없으므로 제목 유사도를 유지합니다. 기간 보고는 조직의 한 업무를 한 줄로 보여 주기 위한 것이기 때문입니다.

### 3.5.17 v0.33 — 개인 API 키 경계 훑기 (완료)

v0.32의 경계 훑기를 세션 쿠키가 아니라 개인 API 키로 다시 했습니다.

- 키의 보안 자세는 견고했습니다. 범위 이름이 전부 `:read` 인 것을 보고 "저장되기만 하고 강제되지 않을 것"이라고 가정했는데 틀렸습니다. `apiKeyRequestAllowed` 가 기본 거부에 GET 전용이고, 허용 경로도 네 갈래뿐입니다. 읽기 전용 키로 수정·제출·생성·키 발급·키 회전을 시도해 전부 403이었습니다.
- 키는 주인을 넘어서지 못합니다. 일반 사용자의 키에 `analytics:read` 를 넣어도 팀 분석은 볼 수 없습니다. 역할 검사가 그대로 뒤에 이어지기 때문입니다.
- 회전은 즉시 듣습니다. 조회 질의가 키 버전을 대조하므로 회전 다음 요청부터 401입니다. 200 → 401을 실제로 확인했습니다.
- 조직 경계도 세션과 같습니다. 조직 1 팀장의 키로 조직 7 보고서 403, 팀 보고서는 260건으로 자기 조직만.
- 다만 거부 이유가 하나뿐이었습니다. 범위가 없어서인지, 경로 자체가 닫혀 있어서인지, 쓰기라서인지가 구분되지 않았습니다. 업무 추적·인수인계·회의 모드·경영 요약·업무 인사이트·담당자 목록은 어떤 범위로도 열리지 않는데, 메시지는 범위를 추가하면 될 것처럼 읽혔습니다. 셋을 구분해 말합니다.
- 이 규칙이 어디에도 적혀 있지 않았습니다. README는 키를 MCP 토큰으로만 소개했고 openapi의 `/keys` post에는 설명이 없었습니다. 범위별로 무엇이 열리는지 표로 적었습니다.

### 3.5.16 v0.32 — 권한 경계 훑기 (완료)

경로 79개를 가드별로 나누면 `requireAuth` 만 47개, `requireRole(ADMIN)` 25개, 팀장 이상 7개입니다. 47개는 핸들러가 스스로 범위를 좁혀야 한다는 뜻이므로, 그쪽을 두드렸습니다.

- 일반 사용자 경계는 전부 막혀 있었습니다. 다른 사람의 보고서 읽기·수정·삭제·제출·복제·댓글·승인·반려, 인수인계, 팀장 전용 5개, 관리자 전용 3개 — 19가지 전부 403이었습니다.
- 팀장의 조직 경계도 막혀 있었습니다. 다른 조직 보고서 403, 업무 병합·분리 400, 팀 보고서·담당자 목록·업무 인사이트 모두 자기 조직으로 좁혀졌습니다.
- 한 곳이 통과했습니다. 인수인계가 역할만 확인하고 대상이 범위 안인지는 보지 않았습니다. 업무는 `loadWorkItems` 가 걸러 새지 않았지만, 이름은 사용자 표에서 그대로 채워 넣고 있었습니다. 팀장이 식별자를 훑어 전사 이름을 읽을 수 있었고, 더 나쁘게는 범위 밖 담당자가 평범한 빈 인수인계로 돌아와 "열어 볼 수 없는 사람"과 "말은 일이 없는 사람"이 구분되지 않았습니다.

- 허용 대상을 `/team/members` 가 제안하는 사람과 **정확히** **같게** 맞췄습니다. 목록이 제안한 사람은 전부 열리고, 목록 밖은 전부 403입니다. 목록이 제안해 놓고 화면이 거절하면 그 목록은 거짓말입니다.

### 3.5.15 v0.31 — 쓰기 표면 훑기, 화면에만 있던 규칙 (완료)

GET 42개에 이어 쓰기 37개를 훑었습니다. 각각 빈 객체·깨진 **JSON·배열·null** 네 가지 잘못된 본문으로 호출했고, 경로 변수는 존재하지 않는 값으로 채웠습니다.

- 표면 자체는 튼튼했습니다. 500이 하나도 없었습니다. `AI_UNAVAILABLE` 503은 AI를 끈 배포에서 맞는 답이고, 개인 키 회전은 본문을 무시하지만 **호출자 본인 키에만** 걸리고 감사 로그도 남으므로 결함이 아닙니다.
- 다만 빈 본문으로 `POST /reports` 를 부르면 항목 없는 보고서가 만들어졌고, 그대로 제출까지 됐습니다. 화면은 "업무 항목을 하나 이상 입력하세요"로 막는데 그 규칙이 **브라우저에만** 있었습니다. API와 MCP는 문서화된 표면입니다.
- 초안은 비어 있어도 됩니다 — 초안이란 그런 것입니다. 제출이 한 주가 끝났다고 말하는 행위이므로 **제출** 지점에서 막습니다. 409 `EMPTY_REPORT` .
- 통합 테스트에 이 경우를 넣고, 가드를 되돌리면 실패하는 것을 실제 데이터베이스에서 확인했습니다.

### 3.5.14 v0.30 — 내보내기가 앱 밖으로 나가 원시 JSON을 보여 주던 문제 (완료)

`asString` 나머지 호출부를 훑었고 더 나올 것이 없었습니다. 남은 네 곳은 모두 `nil` 가드 안이거나 정수 인자였습니다. 대신 v0.29에서 결함을 찾아낸 방식 — 기본 인자로 표면 전체를 훑기 — 을 API 전체에 적용했습니다. GET 42개를 전부 호출했고 5xx는 없었지만, 같은 조건에서 세 엔드포인트가 서로 다르게 답하는 것이 눈에 띄었습니다.

- 내보내기가 전부 `<a href>` 로 **API**를 직접 가리키고 있었습니다. 잘 될 때는 동작하지만 안 될 때가 문제입니다. 브라우저가 앱을 떠나 응답을 그대로 그립니다. 세션이 만료된 탭에서 내보내기를 누르면 `{"success":false,...,"UNAUTHORIZED"}` 가 페이지로 표시되고 앱은 탭에서 사라집니다. 돌아올 방법은 뒤로 가기뿐입니다.
- 내보내기를 `fetch` 로 바꿔 실패가 다른 실패와 같은 토스트로 가고, 401이면 다른 곳과 똑같이 세션 만료 화면으로 갑니다. 서버가 정한 한글 파일명은 `Content-Disposition` 에서 그대로 가져옵니다.
- 비어 있는 기간에서 세 엔드포인트가 서로 다르게 답하고 있었습니다. 목록은 200에 `items: []` , CSV는 200에 머리글만, PPTX는 409 `EMPTY_ROLLUP` . 발표 모드 버튼만 잠겨 있고 CSV·PPTX는 누를 수 있었습니다. 세 버튼을 같은 조건으로 잠급니다.

### 3.5.13 v0.29 — 남은 상한을 끝까지, 그리고 MCP 도구가 첫 질문에 답하지 못하던 문제 (완료)



- **MCP 도구 셋이 인자 없이 부르면 실패하고 있었습니다.** 선택 인자를 `asString(arguments["x"])` 로 읽는데, 없는 키에 `fmt.Sprintf` 를 걸면 빈 문자열이 아니라 문자열 `"<nil>"` 이 나옵니다. 모든 기본값 처리가 `if value == ""` 라 그 네 글자가 실제 값처럼 통과했습니다. `weekly_reports_search` 는 `"<nil>"` 을 날짜로 PostgreSQL에 보냈고, `period_report_rollup` 은 조회 범위가 잘못됐다고 답했습니다. 에이전트가 가장 먼저 던지는 질문 — "무슨 보고서가 있나" — 이 늘 실패하고 있었습니다.
- **MCP 도구 실패 원인을 아무 데도 남기지 않고 있었습니다.** 호출자에게는 한 문장, 고칠 사람에게는 아무 것도 없었습니다. 이 로그를 붙이자마자 위 결함이 드러났습니다.
- 남은 조용한 상한을 정리했습니다. MCP 보고서 검색(`LIMIT 100`)은 전체 건수와 함께 반환하고, 일부만 보냈다는 사실을 문장으로도 적습니다. 이 응답을 읽는 것은 사람이 아니라 언어 모델이고, 모델의 세계는 이 payload가 전부입니다. Import 이력(`LIMIT 100`)은 전체 건수와 쪽 넘기기를 갖습니다.
- 관리자 화면의 "미제출 인원" 숫자가 **25**에서 멈추고 있었습니다. 목록은 누락이 많은 순 상위 25명인데 헤드라인 숫자가 그 목록의 길이였습니다. 41명이 밀려 있어도 25로 보였습니다. 순위는 순위대로 두고 전체 인원을 따로 셉니다.
- **AI 초안 경고가 20건에서 조용히 잘리고 있었습니다.** 몇 건을 버렸는지 적는 줄을 붙입니다.

### 3.5.12 v0.28 — 우리가 만든 규칙을 우리가 어기고 있던 자리 (완료)

v0.25에서 5.의 품질 관리에 "목록을 반환하는 API는 상한과 전체 건수를 함께 갖습니다"를 적었습니다. 정작 보고서 목록이 그 규칙을 어기고 있었습니다.

- `queryReports` 가 `LIMIT 500` 만 걸고 배열을 그대로 돌려주고 있었습니다. 담당자 120명·26주에서 보고서 3,114건 중 500건이 왔고, 화면은 최근 5주만 보여 주면서 그게 전부인 것처럼 보였습니다. `{items, total, limit, offset}` 으로 바꾸고 화면은 **3,114건 중 200건** 과 **더 보기** 를 표시합니다.
- 그 잘림이 인수인계 화면에서 기능 결함이 되고 있었습니다. 담당자 선택 목록을 팀 보고서 한 쪽에서 만들고 있어서, 최근 **5주** 안에 보고하지 않은 사람은 고를 수 없었습니다. 6주 전에 조직을 떠난 사람으로 재현했습니다 — 인수인계가 필요한 사람이 정확히 그 사람입니다. 사람 목록은 사람 표에서 옵니다(`GET /api/v1/team/members`). 비활성 계정도 포함하고 마지막 보고 주차를 함께 보여 줍니다.
- 빠른 이동은 **8건만** 쓰면서 **200건을** 받아 오고 있었습니다. `limit=8` 을 붙였습니다.
- 상한 값은 잘라 맞추고(1~500) 읽을 수 없는 값은 기본값으로 봅니다. 400으로 돌려주는 것은 요청한 쪽이 이미 아는 것을 다시 말할 뿐입니다.

### 3.5.11 v0.27 — 메타포를 남은 화면으로, 색이 갈라진 나머지 자리 (완료)

- 대시보드의 "지난주 대비 변화"가 숫자 일곱 개였습니다. **완료 0 신규 0 재개 3 진척 0 역행 0 정체 0 누락 3** 은 데이터이지 답이 아닙니다. 한 줄 띠로 그려 넓은 주황이면 정체된 주, 넓은 회색이면 보고가 빠



진 주로 읽힙니다. 신규·재개·진척은 모두 "움직였다"는 뜻이라 같은 파랑을 쓰고 읽기 순서상 붙어 있어 한 덩어리로 보입니다.

- **업무 추적 상세도 같은 주차 기록을 갖고 있었습니다.** v0.26의 주차별 띠를 그대로 붙였습니다. 아래 목록은 "N주차에 무슨 일이 있었나"에 답하고, 위 띠는 "몇 주차를 읽어야 하나"에 답합니다. 상세를 여는 사람이 실제로 가진 질문은 후자입니다.
- **색이 갈라진 자리가 세 군데 더 있었습니다.** v0.26이 토큰으로 모은 것은 차트와 결재 배치였고, 대시보드 변화 숫자·업무 상태 칩·증감 표시는 여전히 각자 hex를 썼습니다. 대시보드는 완료를 청록으로, 보고 누락을 빨강(=문제)으로 칠하고 있어 바로 옆에 새로 붙인 띠와 어긋났습니다.
- **.delta-up / .delta-down 이 각각 두 번 정의돼 있었습니다.** 용어 언급 증감(파랑/회색)과 진척 증감(녹색/빨강)이 같은 클래스 이름을 썼고, 뒤에 온 정의가 앞것을 전부 덮었습니다. 워드클라우드에서 늘어난 용어가 줄어든 용어의 색으로 그려지고 있었습니다. 축이 다르므로 이름을 나눴습니다.
- **같은 선택자가 서로 다른 색으로 두 번 정의되면 테스트가 실패합니다.** 색 이름과 값의 표기 차이( `var(--blue)` 와 `#2563eb` )는 충돌로 세지 않습니다.

### 3.5.10 v0.26 — 시각적 메타포 (완료)

화면 15개 중 시각 요소가 있는 것은 2개뿐이었고, 나머지는 표와 문장이었습니다. 표가 답을 담고 있어도 읽는 사람이 답을 조립해야 하면 그 화면은 답한 것이 아닙니다.

- **색이 두 곳에서 반대로 말하고 있었습니다.** 차트에서 주황은 정체, 파랑은 진행인데, 보고서 결재 배치에서는 주황이 제출됨, 파랑이 마감이었습니다. 기간 보고 화면은 둘을 같은 카드에 놓습니다. 한 색이 한 뜻을 갖도록 정리하고(회색 멈춤·파랑 진행·녹색 완료·주황 정체·빨강 문제) 값을 `:root` 한 곳에 모았습니다. 구성비 색은 어느 상태도 쓰지 않는 보라 계열로 분리했습니다. **다시 갈라지면 Go 테스트가 실패합니다.**
- **협업 "지도"가 66행짜리 표였습니다.** 본부 12개면 조합이 정확히 66개입니다. "우리 본부가 어디와 붙어 있나"를 알려면 66줄을 읽고 자기 이름이 나온 줄을 기억해야 했습니다. 조직 × 조직 행렬로 바꿔 가로줄 하나가 곧 그 조직의 이웃이 됩니다. 정확한 숫자가 필요하면 표를 펼칠 수 있습니다.
- **인수인계에 필요한 그림이 이미 만들어져 있었습니다.** 화면 클래스 이름은 `handover-timeline` 인데 실제로는 목록이었습니다. 인수인계가 답할 질문은 "언제부터, 얼마나 자주, 어디서 멈췄나"인데, 진행 경과와 진척이 변한 주차만 남기므로 빈 구간을 표현할 수 없습니다. 주차별 기록을 응답에 담고 보고한 주는 채우고 누락된 주는 비워 그립니다. 26주 중 6주만 보고된 업무가 한눈에 보입니다.
- **경영 요약 점수가 왜 그 순서인지 안 보였습니다.** 점수는 원래 귀속 가능한 사실의 합인데(중복 +40, 이슈 주차 ×10 ...) 응답은 총점 하나와 문장 목록만 보냈습니다. 근거마다 기여 점수를 함께 보내고 화면은 점수를 그 부분들로 이루어진 막대로 그립니다. 막대 길이는 화면 최고점 대비이므로 순위가 읽지 않아도 보입니다. **총점과 근거 합계가 어긋나면 테스트가 실패합니다.**

### 3.5.9 v0.25 — 업무 인사이트 규모 (완료)

- 응답에 상한이 없었습니다. 업무 1,800건에서 조건에 맞는 쌍 234,900개를 전부 담아 **158MB·2.3초**였고, 화면은 그것을 **map** 으로 전부 그리려 했습니다. 관련도 상위 200건씩만 담고 전체 건수를 함께 반환하도록 바꿔 **297KB·0.3초**가 되었습니다. 처음 발견 당시의 더 겹치는 데이터에서는 링크 1,606,500개에 911MB·8.8초였습니다.
- 같은 제목을 **320만 번** 토큰화하고 있었습니다. 비교는 쌍마다 필요하지만 토큰화는 업무마다 한 번이면 됩니다. 미리 계산해 재사용하니 2.5초에서 0.5초가 되었습니다.
- 인수인계가 조직 전체의 비교 결과를 받아 쓰고 있었습니다. 한 사람을 넘겨주는 화면인데 모든 쌍을 비교하고 그중 그 사람 것만 골랐습니다. 대상자에 닿는 쌍만 비교하고 업무당 5건까지 담아 **858KB·1.5초** → **29KB·0.03초**가 되었습니다.
- 잘라 냈다는 사실을 화면이 말합니다. 로드맵의 "silent cap 금지"를 따라, 탭 숫자는 전체 건수를 보여 주고 목록 위에 몇 건 중 몇 건인지 적습니다.
- 집계는 상한 밖에서 합니다. 협업 지도와 경영 요약의 중복 채점은 조건에 맞는 모든 쌍에서 만듭니다. 목록의 상한이 집계의 상한이 되면 화면이 없는 것을 없다고 말하게 됩니다. 커밋 전 리뷰에서 잡았습니다.
- 5.의 품질 관리에 한 줄이 붙습니다. 목록을 반환하는 **API**는 상한과 전체 건수를 함께 갖습니다.

### 3.5.8 v0.24 — 작성 화면과 대화상자 (완료)

측정 결과 매주 여는 화면이 부차적인 것부터 보여 주고 있었습니다.

- 작성 화면 우선순위: 1440×900에서 첫 업무 제목 입력칸이 y=921로 폴드 아래에 있었고, 1366×768에서는 업무 카드 전체가 화면 밖이었습니다. 그 위를 AI 카드(180px, 꺼져 있으면 안내만)와 캡처 카드(138px, 저장 전에는 사용 불가)가 차지했습니다. AI 카드는 실제로 쓸 수 있을 때만 띄우고 캡처는 업무 항목 아래로 옮겨 **y=567**로 올렸습니다.
- 포커스와 저장 상태: 작성 화면이 커서를 아무 데도 놓지 않았고(로그인 화면은 제대로 놓습니다), 저장 여부를 알리는 표시도 없어 v0.16.0의 이탈 확인 대화상자를 만나야 알 수 있었습니다. 둘 다 넣었습니다.
- 대화상자 동작 통일: 상세 창 여섯 개가 Esc로 닫히지 않고 포커스를 잡지 않으며 Tab이 뒤 화면으로 새어나갔습니다. 팔레트와 발표 모드에는 Esc가 있어 같은 앱에서 창 닫는 방법이 화면마다 달랐습니다. 공용 **Modal** 로 모았습니다.
- 빈 상태: 절반은 원인과 해결을 말하고 절반은 사실만 말했습니다. 다음 행동이 있는 곳에는 버튼을 넣었습니다.

### 3.5.7 v0.23 — 설정 안전장치와 감사 추적 (완료)

- **주차 시작 요일 변경:** 서식 설정처럼 보이지만 실제로는 데이터 이관입니다. 실제 서버에서 MONDAY → WEDNESDAY로 바꾼 결과, 작성자의 이번 주 보고서가 사라져 같은 기간에 두 번째 보고서를 만들 수 있었고, 참여 분석의 과거 주차가 전부 0건이 됐으며, 기간 보고가 한 주를 두 주로 쪼갰습니다. 이제 영향 범위를 알리고 확인을 받으며, 기간이 겹치는 보고서 생성 자체를 막습니다.
- **조회 가능한 감사 이력:** 최근 500건을 조건 없이 반환하는 것이 전부였고 보관 정책도 없었습니다. 저장은 하되 답할 수는 없는 통제였습니다. 작업·행위자·대상·기간 필터와 페이지, 전체 건수, 보관 기간 설정을 넣었습니다.
- **오류 추적 ID:** 서버가 요청마다 만들어 로그에 남기고 응답에도 담던 식별자를 화면이 버리고 있었습니다. 5xx 오류 문구에 함께 보여 줍니다.
- 4.3에 한 줄이 붙습니다. 과거 데이터의 의미를 바꾸는 설정은 확인 없이 적용하지 않습니다.

### 3.5.6 v0.22 — 기동과 휴대폰 (완료)

- **기동이 백필을 기다리지 않습니다:** 업무 식별자 백필과 첨부 무결성 확인이 서버가 응답을 시작하기 전에 동기로 돌고 있었습니다. 5만 건에서 `/healthz` 응답까지 34.8초가 걸렸고, 쿠버네티스 기본 `livenessProbe` 는 약 30초에 파드를 죽입니다. 재시작할 때마다 하나의 트랜잭션을 처음부터 다시 시작했으므로 규모가 큰 배포는 영원히 끝내지 못합니다. 배경으로 옮기고 500건씩 나눠 커밋해 재시작에도 이어지게 했습니다. `startupProbe` 도 매니페스트에 넣었습니다.
- **휴대폰에서 메뉴 4개에 닿지 못했습니다:** 하단 메뉴 13개 중 4개가 화면 밖에 그려지고 그 띠는 스크롤되지 않았습니다. 릴리즈를 거듭하며 메뉴를 늘린 결과입니다. 스크롤 가능하게 하고, 가장자리 페이드로 더 있다는 것을 알리고, 현재 화면의 항목이 보이도록 스크롤합니다.
- 4.2의 원칙에 한 줄이 붙습니다. 기동 경로에 말뚝치 전체를 도는 작업을 추가하면 안 됩니다. 필요하면 배경에서, 이어서 할 수 있게 나눠서 합니다.

### 3.5.5 v0.21 — 발표 화면 테마 (완료)

- **읽을 수 있는 최소 글자 크기:** v0.20의 자동 축소는 배율만 줄여서, 1280×720에서 구획 이름이 9.3px 까지 내려갔습니다. 이제 여백을 먼저 줄이고, 글자는 화면 높이의 1/50(프로젝션 경험칙)을 밑돌지 않는 선까지만 줄입니다. 그래도 넘치면 줄이는 대신 잘렸다고 알립니다. 구획 이름의 상한이 화면과 무관한 15px 고정이던 것도 화면에 비례하도록 바꿨습니다.
- **테마 3종:** 어두운 화면(기준), 밝은 화면, 고대비. 밝은 방과 약한 프로젝터에서는 어두운 그라데이션이 가장 불리하고, 캡처 슬라이드는 대개 밝은 스크린샷이라 어두운 판 위에서 눈부십니다. 고대비는 그라데이션 없이 검정 바탕에 단일 강조색만 씁니다. 약한 프로젝터가 여섯 색을 구분해 줄 것이라고 기대할 수 없기 때문입니다.

- **회사 템플릿 강조색 연동:** 내보낸 PPTX는 회사 템플릿을, 화면은 고정 파랑·청록을 써서 같은 보고가 매체에 따라 다른 회사 자료처럼 보였습니다. 템플릿 테마의 `accent1` 을 읽어 화면 강조색으로 씁니다.
- 구획 색이 없던 `변경점` 에 색을 넣고, 발표자 창이 테마를 따르게 하고, `prefers-reduced-motion` 을 존중합니다.

### 3.5.4 v0.20 — 발표 모드 (완료)

v0.13에서 회의용으로 만든 발표 모드를 실제 회의실 화면 크기에서 다시 점검했습니다.

- **내용이 잘리던 문제:** 슬라이드 텍스트 상한이 화면 크기와 무관한 고정 글자 수였고, 슬라이드 상자는 `overflow:hidden` 이었습니다. 1280×720에서 한 블록의 **39%**가 아무 표시 없이 사라졌습니다. 이제 상자를 실제로 재서 들어갈 만큼만 줄입니다. 읽을 수 있는 하한(0.62배)을 넘어서까지 줄이지는 않고, 그 때만 안내를 띄웁니다.
- **발표자 화면:** 회의는 보통 한 화면을 방에 미러링하고 노트북은 발표자가 봅니다. 두 화면에 다른 것이 있어야 합니다. 별도 창에 경과 시간, 다음 슬라이드, 그리고 **슬라이드가 줄여서 보여 주는 그 내용의 전체 원문**을 띄웁니다. 측정값으로 881자 대 2292자였습니다.
- **슬라이드 목록과 번호 이동:** 40장짜리 데크에서 "그건 3번 안건이었죠"에 대응할 방법이 화살표뿐이었습니다.
- 화면 끄기(B), 태블릿 스와이프, 새로고침 후 위치 복원, 스크린리더 안내를 함께 넣었습니다.

### 3.5.3 v0.19 — 내보내기 안전성과 실패 복구 (완료)

- **CSV 수식 주입 차단:** 기간 보고 CSV가 보고 본문을 그대로 실어 보내고 있었습니다. 스프레드시트는 `=, +, -, @` 로 시작하는 셀을 수식으로 실행하므로, 아무 직원이 쓴 `=HYPERLINK(...)` 가 그 파일을 여는 상급자의 기계에서 실행됩니다. 셀 앞에 작은따옴표를 붙여 막았습니다. `-` 뒤에 공백이 오는 글머리표는 그대로 둡니다. 공격은 연산자 바로 뒤에 함수 이름이나 명령이 있어야 성립하고, 그렇지 않으면 보고서 본문 상당수에 따옴표가 붙습니다.
- **세션 만료 복구:** 세션이 끊겨도 화면이 그대로 돌아가고 있었습니다. `.catch` 가 없는 화면은 스피너가 영원히 멈추지 않았고, 어디에도 401을 처리하는 곳이 없었습니다. API 계층이 401을 알리고 셀이 로그인 화면으로 되돌립니다. 단, **저장하지 않은 편집이 있으면 화면을 유지합니다.** v0.16.0이 지키기로 한 바로 그 내용을 로그인 폼을 띄우려고 버릴 수는 없습니다. 세션 쿠키는 탭 사이에 공유되므로 다른 탭에서 로그인하면 이 화면 그대로 저장됩니다.
- **화면 오류 격리:** 렌더 중 예외 하나가 앱 전체를 흰 화면으로 만들었습니다. 오류 경계를 넣어 해당 화면 안에서 끝나게 하고 돌아갈 길을 제시합니다.

- **PPTX 내보내기 메모리 축소:** 첨부 이미지를 전부 메모리에 올린 뒤, 본문 앞·뒤 두 묶음 때문에 패키지 전체를 두 번 다시 만들고 있었습니다. 한 번에 처리하고 이미지는 zip에 쓰는 순간 읽도록 바꿨습니다.

### 3.6 v0.16 — 편집 보호와 자동 판정 정정 (완료), Decision Log (남음)

v0.16.0은 Decision Log보다 먼저 처리해야 할 세 가지를 처리했습니다. 하나는 지금도 데이터를 잃고 있었고, 하나는 4.3의 원칙이 유일하게 지켜지지 않던 자리였으며, 하나는 보안 통제 목록에서 유일하게 비어 있던 항목이었습니다.

- **편집 내용 유실 방지 (완료):** 저장하지 않은 편집이 있으면 화면 이동·뒤로 가기·탭 닫기·로그아웃 전에 확인합니다. 저장에 실패했을 때 화면 내용을 서버 사본으로 덮어쓰던 동작도 제거했습니다. 실패한 저장이 지키려던 바로 그 내용을 지우고 있었습니다.
- **업무 병합·분리 (완료):** 3.1이 약속하고 4.3이 요구한 정정 경로입니다. 자동 판정을 되돌릴 수단이 없으면 그것은 제안이 아니라 확정이며, 오판은 Aging·인수인계·중복 탐지·협업 지도까지 전파됩니다.
- **로그인 시도 제한 (완료):** 시도 횟수 제한이 전혀 없었습니다. 계정당 제한은 기본 활성화, IP당 제한은 기본 비활성입니다. NAT 뒤에서는 의미 있는 임계값이 무관한 사용자를 함께 잠급니다.

정정이 다음 저장에도 살아남는 것이 이 기능의 전부입니다. 업무 식별은 저장할 때마다 제목에서 다시 유도되므로, 유도 결과를 바꾸지 않는 정정은 작성자의 다음 편집이 되돌립니다. 병합은 원본 행과 정규화 키를 남기고 대상을 가리켜 옛 제목이 대상으로 해석되게 하고, 분리는 옮긴 스냅샷을 고정해 제목이 바뀌기 전까지 유도를 이깁니다.

#### 3.6.1 v0.34 — Decision Log 1단계: 명시적 기록 (완료)

로드맵이 순서를 못 박아 뒀습니다 — **명시적 기록이 먼저, AI 제안이 나중**. 감사·기억 시스템의 신뢰도를 모델 재현율에 걸 수 없기 때문입니다. 이번에는 앞의 절반을 만들었습니다.

- **주간보고가 담지 못하는 기록입니다.** 보고서는 무슨 일이 있었는지를 적지, 누가 무엇을 왜 정했고 무엇을 하기로 했는지는 적지 않습니다. 그래서 반년 뒤에 실제로 나오는 질문 "왜 이렇게 하기로 했더라"는 아직 기억하는 사람을 찾아 물어야 했고, 아무도 기억하지 못하는 순간을 위해 만든 인수인계 화면에 보여 줄 것이 없었습니다.
- **결정자와 기록자를 구분합니다.** 팀장이 임원 결정을 대신 적는 경우가 흔합니다. 둘을 합치면 이 기록이 가장 필요한 상황에서 값이 틀립니다.
- **뒤집힌 결정을 지우지 않습니다.** 새 결정이 이전 결정을 대체하면 이전 것은 **SUPERSEDED** 가 됩니다. 뒤집힌 결정은 없던 결정보다 정보가 많습니다.

- 기록은 볼 수 있는 사람이, 삭제는 기록한 사람이. 업무 병합·분리와 규칙이 다른데, 병합은 남의 보고 이력을 고쳐 쓰는 일이고 기록은 덧붙이는 일이기 때문입니다. 반대로 다른 사람이 조용히 지을 수 있는 기록은 기록이 아닙니다.

남은 것은 3.6.2를 보십시오.

### 3.6.2 v0.35 — Decision Log 2단계: 인수인계의 결정 이력 (완료)

이 기록이 필요한 이유가 인수인계였습니다. 업무 추적에만 있으면 결정을 적은 사람만 볼 수 있고, 정작 아무 것도 모르는 채 업무를 넘겨받는 사람에게는 닿지 않습니다.

- 잘라 내지 않습니다. 인수인계가 존재하는 이유가 "아무도 기억하지 못하는 것"인데 그 기록을 잘라 내면 화면의 존재 이유가 사라집니다. 업무 15건·결정 17건에서 16.0KB → 22.9KB, 8ms → 5ms였습니다.
- 질의는 하나입니다. 업무마다 조회하면 화면에 뜬 업무 수만큼 질의가 나옵니다. v0.25가 한 릴리즈를 들여 없앤 모양입니다.
- 기한 지난 후속 조치가 주의 문구를 이깁니다. 다른 주의 문구는 기록이 무엇을 보여 주는지 말하지만, 이것은 누군가 약속하고 지키지 않은 것을 지목합니다. 머리줄에 미해결 결정 수와 기한 초과 수를 함께 셉니다.
- 인수인계에서는 읽기만 합니다. 인수인계는 읽는 문서이지 일하는 화면이 아닙니다. 기록과 정정은 업무를 관리하는 업무 추적에 둡니다.

### 3.6.3 v0.36 — 회의 결과 반영 (완료)

회의는 안건을 만들어 주고 거기서 멈췄습니다. 회의에서 나온 결론은 누군가의 수첩으로 들어갔고, 다음 월요일에 그 업무의 담당자는 아무 표시도 없는 빈 편집기를 열었습니다.

- 회의 화면에서 그 자리에 적습니다. 안건이 화면에 떠 있고 문구가 아직 생생할 때 적는 것이 가장 정확합니다. 나중에 옮겨 적으려면 대개 옮겨 적지 않습니다.
- 후속 조치가 주인에게 돌아옵니다. 작성 화면이 지난주 계획을 돌려주는 바로 그 자리에 결정에 딸린 후속 조치 카드가 생깁니다. 기한이 지난 것은 그렇게 표시합니다.
- 차주 계획으로 들어갑니다. 금주 실적이 아닙니다. 하기로 한 일이지 했다는 주장이 아닙니다.
- 이미 이번 주 보고에 쓴 업무는 빼고 보여 줍니다. 작성자가 이미 적은 것을 다시 권하면 카드 전체가 무시됩니다.
- 본인 업무만입니다. 후속 조치는 해야 할 일이고, 남의 의무 목록은 할 일 목록이 아닙니다.

### 3.6.4 v0.37 — 기간 보고의 결정 이력 (완료)

로드맵의 결정 이력 조회가 이걸로 끝납니다. 인수인계(v0.35)에 이어 기간 보고에도 붙였습니다.

- 무엇을 했는지 위에 있고, 왜 그렇게 하기로 했는지가 아래에 있습니다. 분기 보고가 일한 목록만 담고 결정을 빼면, 읽는 사람은 왜 그렇게 갔는지 물을 수 없습니다.
- 사람 범위 조건을 집계와 공유합니다. 조건을 두 벌 쓰면 언젠가 어긋나고, 한쪽 사람들을 집계하면서 다른 쪽 결정을 나열하는 보고서는 결정을 아예 안 넣느니만 못합니다. 조직 1 팀장 17건 / 관리자 18건으로 범위가 갈리는 것을 확인했습니다.
- 상한 50건에 전체 건수를 함께 줍니다. 5.의 "목록을 반환하는 API는 상한과 전체 건수를 함께 갖는다"를 따릅니다.

### 3.6.5 v0.38 — 결정을 텍과 발표 모드로 (완료)

v0.37은 화면만 채웠습니다. 임원이 실제로 받는 것은 화면이 아니라 텍입니다.

- PPTX에 기간 내 결정 슬라이드가 들어갑니다. 업무 상세 뒤, 이슈·리스크 앞입니다 — 기간이 스스로를 설명한 뒤 요청을 합니다. 끝을 요청으로 맺어야 회의가 움직입니다.
- 후속 조치가 남은 결정을 먼저 씁니다. 이미 이행된 결정은 이력이고, 아직 값을 것이 남은 결정이 브리핑에 들어갈 이유입니다.
- 발표 모드도 같이 받습니다. 3.4가 정한 원칙입니다 — 내보내기와 발표는 같은 내용을 다른 매체로 전달하므로 한쪽만 제공할 이유가 없습니다.
- 표 칸에 줄바꿈을 쓰지 않았습니다. DrawingML에서 `<a:t>` 안의 줄바꿈은 줄바꿈이 아니라 `<a:br/>` 이 필요합니다. 확인할 수 없는 렌더링 동작에 기대는 대신 구분자를 명시했습니다.

### 3.6.6 v0.39 — AI 후보 제안 (완료), Decision Log 완결

로드맵이 이 기능을 마지막에 둔 이유를 그대로 따랐습니다. 감사·기억 시스템의 신뢰도를 모델 재현율에 의존시킬 수 없으므로, 명시적 기록(v0.34)·인수인계(v0.35)·회의 반영(v0.36)·기간 보고(v0.37·v0.38)가 모두 섰 뒤에야 그 위에 얹었습니다.

- 제안만 합니다. 아무것도 저장하지 않습니다. 후보는 사람이 고쳐서 확정할 양식일 뿐이고, 실제로 0건이 저장되는 것을 확인했습니다.
- "여기 없다고 해서 결정이 없었다는 뜻이 아니다"를 서버가 함께 보냅니다. 화면이 빠뜨릴 수 없도록 payload에 담습니다. 전수인 것처럼 보이지만 아닌 목록은 목록이 없는 것보다 나쁩니다.
- 후보마다 보고 원문을 인용합니다. 확정하는 사람이 모델이 아니라 기록에 대고 확인할 수 있어야 합니다.
- 모델이 지어낸 날짜는 서버가 비웁니다. 없는 날짜가 그럴듯한 날짜보다 낫습니다 — 비어 있으면 양식이 오늘로 채우고, 확정하는 사람이 알아차립니다.



- **AI가 꺼진 배포에서는 버튼을 띄우지 않습니다.** 늘 "관리자가 설정해야 합니다"라고 답하는 버튼은 사람에게 버튼 누르기를 그만두라고 가르칩니다.

이로써 3.6.1의 Decision Log 네 항목이 모두 완료됐습니다.

- **명시적 기록 우선:** 결정사항을 항목 단위로 직접 기록하는 필드를 먼저 제공합니다. 결정자, 날짜, 근거, 후속 조치, 관련 WorkItem을 구조화합니다.
- **AI 후보 제안:** 기존 보고 텍스트에서 결정사항 후보를 제안하고 사용자가 확정합니다. 감사·기억 시스템의 신뢰도를 모델 재현율에 의존시키지 않습니다.
- **결정 이력 조회:** WorkItem 타임라인과 기간 보고에서 결정 흐름을 함께 확인합니다.
- **회의 결과 반영:** 회의 모드는 v0.13에서 완료됐고, 남은 것은 결과를 되돌려 쓰는 것입니다. 회의에서 정한 Decision과 Action Item을 기록하면 다음 주 보고서 초안에 반영합니다. 결정 기록 모델을 전제하므로 이 단계에 함께 둡니다.

### 3.7 v0.40 — Dependency / Blocker Graph 1단계: 관계 모델과 순환 방지 (완료)

- 간선은 하나입니다. "A가 B를 기다린다"와 "B가 A를 막는다"는 같은 사실을 양끝에서 본 것입니다. 두 종류로 저장하면 둘이 어긋날 수 있고, 스스로 모순되는 그래프는 없느니만 못합니다.
- 선언은 한쪽이 합니다. 기다리는 쪽 담당자가 자기 일이 무엇을 기다리는지 선언하고, 상대의 동의를 받지 않습니다. 상대가 동의해야 하는 의존은 아무도 기록하지 않는 의존입니다. 대신 사유를 함께 남기고, 상대는 자기 화면에서 그것을 보고 이의를 제기하거나 관계를 삭제할 수 있습니다.
- 조직 경계를 넘습니다. 선행 업무는 조회 범위 밖이어도 됩니다. 다른 조직의 업무를 기다리는 것이 이 기능이 존재하는 이유입니다.
- 순환은 막고 사슬을 알려 줍니다. 순환 의존이 됩니다:  $A \rightarrow B \rightarrow C \rightarrow (\text{이 업무})$ . 보이지 않는 그래프를 두고 추측하게 두지 않습니다.
- 순환 검사 질의를 실제 PostgreSQL에 PREPARE 하다가 결함을 잡았습니다. 재귀 CTE의 비재귀항이 `varchar(240)[]`, 재귀항이 `varchar[]`여서 PostgreSQL이 질의 자체를 거부했습니다. 양쪽을 `text`로 맞췄습니다.

남은 것: 병목 집계와 회의 모드의 타 조직 Dependency 구획.

#### 3.7.1 v0.41 — 병목 집계와 회의 구획 (완료), 3.7 완결

- **병목 탭:** 미완료 업무 2건 이상을 막고 있는 선행 업무를 셉니다. 로드맵의 "현재 N개 업무가 하나의 선행 항목 때문에 정체"가 이것입니다.

- 범위를 막는 쪽에 걸었다가 되돌렸습니다. 처음에는 선행 업무의 주인을 기준으로 걸렀는데, 그러면 팀장이 자기 팀을 막고 있는 다른 조직의 병목을 못 봅니다 — 정작 가장 필요한 것인데요. 막히는 쪽 기준으로 바꾸니 조직 1 팀장이 조직 7의 업무가 자기 팀 4건을 막고 있다는 것을 봅니다.
- 회의 모드에 **타 조직 대기** 구획을 넣었습니다. 지속 이슈 뒤, 변경점 앞입니다 — 방이 들어야 할 것이 아니라 방이 처리해야 할 것이기 때문입니다. 한 팀 안의 의존은 그 두 사람이 정리하지만, 조직을 넘는 의존은 양쪽에 말할 수 있는 사람이 방에 있어야 풀립니다.
- 완료된 선행 업무는 제외합니다. 끝난 업무는 무엇도 막지 않으며, 남겨 두면 아무도 지우지 않은 관계로 목록이 채워집니다.

이로써 3.7이 완결됐습니다.

### 3.7.2 v0.42 — 선행 관계를 정체 판정에 반영 (완료)

- 원인이 등록된 정체는 다른 안건입니다. 점수는 같습니다 — 일이 멈춘 정도는 같으니까요. 읽는 사람이 할 일이 다릅니다: **진척 정체** 를 보면 담당자에게 묻고, **타 조직 대기** 를 보면 두 조직을 연결합니다. 같은 멈춤에 다른 조치이므로 같게 읽혀서는 안 됩니다.
- 근거 문장이 무엇을 기다리는지 이름을 댁니다. **진척이 2주째 멈춰 있고, 본부 7의 '설비 자동화 구축'(담당자 7)이(가) 끝나기를 기다리고 있습니다.**
- 경영 요약의 **headline** 을 서버가 계산해 두고 어느 화면도 쓰지 않고 있었습니다. 항목마다 **장기 이슈 · 진척 정체 · 중복 투자 의심** 을 정해 놓고 화면은 넷을 전부 **위험** 으로만 보여 줬습니다. v0.23의 추적 ID 와 같은 부류입니다.

### 3.7.3 v0.43 — 보고 품질 점검에 선행 관계 반영 (완료), 3.7 전부 완결

**PLAN\_REPEATED** 는 막힌 이유를 적으라고 요구하는 규칙입니다. 선행 업무를 이미 등록해 둔 사람에게 그 말을 되풀이하는 것은, v0.17이 상위 조직 요청에 대해 세운 원칙을 어기는 것입니다 — **조치한 사람에게 조치 하라고 반복하면 점검 전체가 무시됩니다.**

- 선행 관계가 등록된 업무는 **WARN**이 아니라 **INFO**가 되고, 문구가 "무엇을 기다리는지 이름을 대며 그 관계가 아직 유효한지" 묻는 것으로 바뀝니다.
- 소견 자체는 남깁니다. 계획을 몇 주째 그대로 적은 사실은 여전히 볼 만하고, 등록해 둔 의존이 이미 해소 됐는데 관계만 남아 있는 경우가 실제로 있습니다.

이로써 3.7이 전부 끝났습니다.

- 관계 모델: WorkItem 간 **dependsOn** , **blockedBy** 를 조직 경계를 넘어 관리합니다.
- 병목 집계: "현재 N개 업무가 하나의 선행 항목 때문에 정체" 형태로 경영 화면에 제시합니다.

- **순환 방지:** 관계 등록 시 순환 의존을 차단하고 경로를 함께 보여 줍니다.
- 회의 모드의 **타 조직 Dependency** 안건 구획은 이 단계에서 추가합니다.

### 3.8 v0.44 — Evidence Lineage 1단계: 공통 모델과 PPTX 계보 (완료)

근거는 있었는데, 쓸모 있어지는 순간에 버려지고 있었습니다. Confluence 후보는 `candidate_sources` 에 페이지를, PPTX Import는 분석 결과에 슬라이드 번호를 들고 있었지만, 후보를 수락하거나 Import를 확정하면 `report_items` 행만 남고 둘 다 사라졌습니다. `report_items` 를 참조하는 테이블이 임베딩 하나뿐이었으므로 답이 숨어 있을 곳도 없었습니다.

- 출처 종류마다 표를 만들지 않았습니다. `report_item_sources` 하나에 MANUAL·CONFLUENCE·PPTX·AI\_TEXT·JIRA·GIT·CI·ITSM·API를 담습니다. 커넥터마다 표를 두면 "근거가 몇 건인가"가 종류마다 다른 질의가 되고, 비교할 수 있게 하는 것이 이 모델의 목적입니다.
- 확정 화면이 슬라이드 번호를 되돌려 보냅니다. 분석은 알고 있었고 확정이 버리고 있었습니다.
- 비어 있음은 근거 없음이 아닙니다. 직접 타이핑했거나 이 모델 이전에 작성됐다는 뜻입니다.
- 통합 테스트가 실제 PostgreSQL에서 근거 행을 확인하고, 기록을 빼면 실패합니다.

남은 것: Confluence 수락 경로, 문장 단위 근거, AI 요약의 근거 수 표기.

#### 3.8.1 v0.45 — Confluence 계보 (완료)

- 수락 처리가 항목을 만들지 않고 있었습니다. `acceptConfluenceCandidates` 는 후보에 표시만 하고, 실제 항목은 편집기 저장이 만듭니다. 그리고 편집기가 실어 보내던 `candidateId` 를 서버가 무시하고 있었습니다 — 그래서 근거를 옮길 지점 자체가 없었습니다.
- 서버가 `candidateId` 를 받아 저장 시점에 그 초안의 페이지·버전·활동 종류를 계보 모델로 복사합니다. 저장할 때만 하고 수정할 때는 하지 않습니다 — 이미 저장된 줄을 고친다고 출처가 바뀌지는 않고, 매번 기록하면 한 페이지가 타이핑 횟수만큼 늘어납니다.
- 식별자는 클라이언트가 보내므로 본인 초안인지 서버가 확인합니다.
- 통합 테스트가 실제 PostgreSQL에서 `CONFLUENCE · 설계 검토 회의록 · v7 · 작성 후 수정` 을 확인하고, 복사를 빼면 실패합니다.

#### 3.8.2 v0.46 — 역방향 계보 (완료), 문장 단위는 보류

문장 단위 근거를 만들려다 전제가 없다는 것을 확인하고 멈췄습니다. AI는 내용을 원자 단위 줄 배열 (`currentResults` , `nextPlans` , `issues` )로 나누지만 `sourceSlides` 는 항목 단위입니다. 문장마다 근거를 표시하려면 그 귀속을 모델에게 새로 요구해야 하고, 여기서는 그 귀속의 품질을 측정할 방법이 없습니

다. 측정할 수 없는 기능을 로드맵 항목을 지우기 위해 만드는 것은 이 프로젝트가 지켜 온 방식이 아닙니다. 보류하고 사유를 남깁니다.

대신 계보 모델이 이미 가능하게 만든 것을 만들었습니다.

- **역방향 계보:** 이 페이지가 어느 보고에 근거로 쓰였나. 정방향은 보고서를 읽는 사람에게 답하고, 역방향은 페이지·텍·커밋의 주인에게 답합니다 — 무엇이 자기 것 위에 세워졌는지 가장 늦게 알고, 바꾸기 전에 가장 알아야 하는 사람입니다. v0.44에서 이 질문을 위한 색인을 만들어 놓고 물어볼 방법이 없었습니다.
- 보고서 상세의 근거 칩을 누르면 같은 근거를 쓴 다른 보고가 펼쳐집니다. 조회 범위 안의 인용만 나열합니다.
- 범위 술어에 별칭을 대려고 `work_items` 를 조인했다가, `work_item_id` 가 NULL이면 그 사람의 모든 업무와 조인돼 건수가 부풀려지는 것을 발견하고 보고서 자체를 별칭으로 바꿨습니다.

### 3.8.3 남음 — 문장 단위 근거와 AI 요약 근거 수

- Confluence 후보의 `candidate_sources` 근거 모델을 모든 Source로 일반화합니다. Manual, Confluence, Jira, Git Commit, CI, ITSM, PPTX Import를 같은 provenance 테이블로 관리합니다.
- 문장 단위로 근거 수와 출처 종류를 표시하고 AI 요약에도 근거 수를 함께 표기합니다.

## 3.9 v0.19 — 다중 PPTX 템플릿과 미제출 알림

- 조직별 PPTX 템플릿 매핑과 선택. 기간 보고 전용 레이아웃과 병행합니다.
- v0.10.0의 미제출 현황을 Slack·Teams·메일 알림으로 연결합니다. 오프라인망 제약을 고려해 발송 채널은 관리자 설정으로 선택합니다.

### #### 3.9.1 v0.47.0에서 완료 — 조직별 PPTX 템플릿

배포 하나에 표지 하나였습니다. 본부 두 곳이 서로 다른 보고 서식을 쓰기 시작하면, 그때부터 모두가 남의 표지로 보고서를 내보냅니다.

- `pptx_templates` 에 `organization_id` 를 추가했습니다(마이그레이션 017). 조직당 하나, 전사 기본 하나를 부분 유니크 인덱스 두 개로 보장합니다. 기존 단일 행은 `organization_id IS NULL` 인 전사 기본이 됩니다.
- 내보내기는 작성자가 속한 조직의 서식을 찾습니다. 없으면 가장 가까운 상위 조직으로 올라가고, 끝까지 없으면 전사 기본을 씁니다. 팀마다 같은 파일을 올리게 하면 그중 절반은 작년 판을 쓰게 됩니다.
- 관리자 화면은 적용 대상을 고르고, 그 조직에 실제로 적용되는 서식을 이름까지 보여줍니다. 물려받은 경우 어느 조직에서 왔는지 함께 표시합니다.

발견한 것:

- 전사 기본 조회가 `WHERE id=1` 로 남아 있었습니다. 식별자가 시퀀스가 되면서 새로 올린 기본 서식은 id 가 1이 아니게 됐고, 업로드는 성공하는데 화면은 계속 참조 서식을 보여줬습니다. 조회를 `organization_id IS NULL` 로 바꿨습니다.
- 화면이 하위 팀에 대해 "물려받음"이라고만 답하고 무엇을 물려받는지는 비워 뒀습니다. 관리자가 그 화면 을 여는 이유가 바로 그것이므로, 내보내기가 하는 것과 같은 조상 탐색을 화면도 하게 했습니다.
- 통합 테스트의 정리를 `t.Cleanup` 으로 걸었더니 `defer db.Close()` 뒤에 실행돼 닫힌 풀에 대고 삭제 했습니다. 행이 그대로 남아 다음 실행이 유니크 제약에 걸렸습니다. `defer` 로 바꿨습니다.

#### #### 3.9.2 남음 — 미제출 알림

Slack·Teams·메일 발송은 이 오프라인 환경에서 보냈다는 사실을 확인할 방법이 없습니다. 발송 코드를 쓰는 것은 가능하지만 검증되지 않은 기능을 완료로 표시하지 않기 위해 남겨 둡니다.

### 3.10 v0.20 — Risk Forecast

- Executive Digest는 v0.13에서 완료됐습니다. 남은 것은 **예측**입니다. 진척 추이, 이월 횟수, 이슈 지속 기간으로 일정 위험을 예측합니다.
- 현재 경영 요약은 이미 관측된 사실만 제시합니다. 예측은 성격이 다르므로 근거를 항상 함께 제시하고 근거 없는 점수는 표시하지 않습니다.

#### #### 3.10.1 v0.48.0에서 완료 — 완료 시점 추정

먼저 확인한 사실이 계획을 바꿨습니다. **예측할 일정이 없습니다.** `work_items.due_date` 는 스키마에 있지만 이를 쓰는 API도 화면도 한 군데도 없어서, 전부 NULL입니다. 그래서 마감일 대비 위험을 계산하는 대신, 보고된 진척이 실제로 말하는 것만 계산합니다. "이 속도면 얼마나 더 걸리는가"입니다.

점쟁이가 되지 않도록 두 가지 규칙을 두었습니다.

- **점이 아니라 구간을 냅니다.** 전체 기간 평균 속도와 최근 3주 속도는 대개 다르고, 그 차이가 곧 불확실성입니다. 둘을 하나로 합쳐 신뢰도 점수를 붙이면 읽는 사람이 봐야 할 것이 사라집니다. 구간이 넓다는 것은 "아직 아무도 모른다"를 소리 내어 말하는 것입니다.
- **말하지 않는 쪽을 택합니다.** 2주치는 어떤 두 숫자에도 직선을 그을 수 있으므로 추정하지 않습니다. 진척이 늘지 않은 업무에는 완료 시점이 아예 없다고 답합니다. 1년이 넘는 추정은 주 단위 숫자를 내지 않습니다.
- 모든 추정은 계산 근거인 두 속도와 사용한 주차 수를 함께 표시합니다. 근거 없는 점수는 표시하지 않는다는 원칙을 형태로 강제한 것입니다.

새 발견 하나가 나옵니다. "진행 중이지만 끝나는 시점이 보이지 않는 업무" — 매주 1%씩 꾸준히 올라가는 업무는 정체도 아니고 이슈도 없어서 어느 현황판에서도 정상으로 보이지만, 자기 숫자로는 1년이 넘게 걸립니다. 기존 정체 규칙과 겹치지 않도록 정체 업무는 이 목록에서 제외합니다. 같은 업무를 두 제목 아래 두 번 적는 것이 위험 목록을 안 읽히게 만드는 방법입니다.

실측: 시드 데이터에 주당 1%씩 6주 진행한 업무를 넣자 정체·이슈 규칙은 모두 침묵했고, 이 규칙만 잡아냈습니다.

#### #### 3.10.2 v0.49.0에서 완료 — 업무 마감일과 마감 대비 전망

v0.48은 "얼마나 더 걸리는가"까지만 답할 수 있었습니다. 늦는지 아닌지는 여전히 말할 수 없었는데, 비교할 마감일이 없었기 때문입니다.

- `work_items.due_date` 에 값을 넣을 수 있게 했습니다. `PUT /api/v1/work-items/{id}/due-date` 와 업무 추적 화면의 입력란입니다. 빈 문자열로 해제할 수 있습니다. 잘못 넣은 날짜를 다른 잘못된 날짜로만 바꿀 수 있다면 없느니만 못합니다.
- 마감일이 있으면 그 날짜에 몇 %에 닿는지를 두 속도로 각각 계산합니다. 둘 다 100%에 닿으면 **기한 내 예상**, 둘 다 못 닿으면 **기한 초과 예상**, 한쪽만 닿으면 **기한 불확실** 입니다. 엇갈림 자체가 결론이므로 한 쪽으로 합치지 않습니다.
- 추정의 기준점은 **마지막으로 보고된 주차**이지 오늘이 아닙니다. 한 달째 보고가 없는 업무를 오늘부터 계산하면 일하지 않은 4주를 실적으로 쳐주는 셈입니다.
- 마감일이 지났으면 아무것도 추정하지 않고 지났다고만 말합니다. 관측된 사실에 예측 문장을 붙일 이유가 없습니다.

이전에 제품이 가진 유일한 기한은 결정의 후속 조치 기한이었고, 그것은 **지난 뒤에만** 보고됐습니다. 16일에 15일을 놓쳤다고 알려 주는 것은 경고가 아니라 기록입니다.

발견한 것: 마감일을 저장한 직후 전망이 비어 있었습니다. 열려 있는 상세 대화상자에 `dueDate` 만 덧씌우고 `dueOutlook` 은 저장 전 값을 그대로 두었기 때문입니다. 마감일을 넣는 이유가 바로 그 답을 보려는 것이므로, 저장 후 목록을 다시 읽어 열려 있는 항목까지 교체합니다.

#### #### 3.10.3 v0.50.0에서 완료 — 회의에서 합의된 기한을 마감일로

v0.49에서 마감일을 넣을 수 있게 만들었지만, 넣을 곳을 만들었다고 사람들이 채우지는 않습니다.

`due_date` 가 그동안 비어 있던 이유가 "입력할 곳이 없어서"였다면, 이제 남는 이유는 "채울 이유가 없어서"입니다.

그런데 기한은 원래 담당자가 정하지 않습니다. **회의에서 합의되고, 이 제품은 이미 그것을 기록하고 있습니다** — 결정의 후속 조치 기한입니다. 팀이 모여 "9월 21일까지"를 정하고 결정으로 적어도, 같은 업무의 마감

일 칸은 비어 있고 전망은 마감일이 없다고 답했습니다. 하나의 업무에 두 개의 날짜가 서로 다른 곳에 들어가 서로를 모르는 상태였습니다.

- 마감일이 없는 미완료 업무에 한해, 그 업무에 걸린 **열린 결정의 후속 기한 중 가장 이른 것**을 제안합니다. 가장 이른 것인 이유는 일정을 제약하는 것이 가장 가까운 약속이기 때문입니다.
- **복사하지 않고 제안합니다.** 후속 조치가 업무 전체와 같다는 보장이 없으므로, 조용히 마감일로 승격시키면 회의가 하지 않은 말을 하는 것이 됩니다. 날짜·결정자·후속 조치 문구를 그대로 보여 주고 한 번의 클릭으로 지정합니다.
- 이미 마감일이 있는 업무에는 제안하지 않습니다. 답이 있는 자리에 두 번째 날짜를 나란히 놓으면 답이 논쟁이 됩니다.

#### #### 3.10.4 v0.51.0에서 완료 — 마감 위험을 경영 요약으로

v0.49·v0.50에서 만든 마감 위험은 업무 추적 화면에만 있었습니다. 그런데 기한을 실제로 처리하는 사람은 그 화면을 열지 않고 경영 요약을 봅니다.

경영 요약의 점수는 관측된 사실의 합이고 모든 항목이 근거를 되돌려 줍니다. 마감 위험을 여기 넣으면서 두 반쪽을 다르게 다뤘습니다.

- **기한 초과**는 관측입니다. 날짜가 지났고 완료되지 않았다는 것은 계산이 아니라 사실이므로, 얼마나 오래 그랬는지에 따라 점수가 올라갑니다.
- **기한 초과 예상**은 산술입니다. 그래서 고정 점수를 줍니다. 추정치의 크기가 순위를 끌게 두면 추정이 관측 위에 서게 됩니다.
- **기한 불확실**(두 속도가 엇갈리는 경우)은 넣지 않았습니다. 업무 추적 화면에서는 양쪽을 다 볼 수 있으니 유효한 발견이지만, 브리핑에서는 "아마도"이고 아마도로 가득한 브리핑은 읽히지 않게 됩니다.

실측으로 확인했습니다. 마감일이 없을 때 이 업무는 경영 요약에 아예 없다가(5건), 6주 뒤 기한을 넣으면 25점으로 맨 아래에 붙고(6건), 이미 지난 기한을 넣으면 40점으로 2위에 올라옵니다.

발견한 것: `summarizeWorkItem` 이 멍등이 아니었습니다. 이슈·정체·계획 반복 카운터를 0으로 되돌리지 않고 `++` 로 누적해서, 같은 뷰에 두 번 부르면 매주 진척이 오른 업무가 정체로 바뀝니다. 실제 경로에서는 한 번만 부르므로 드러나지 않았고, 이 회차의 테스트가 두 번 부르면서 걸렸습니다.

#### #### 3.10.5 v0.52.0에서 발견 — 규모에서 드러난 것

작은 데이터로는 전부 빠릅니다. 300명·52주·109,200건으로 실제 인스턴스를 만들어 API 전체를 훑었더니 두 가지가 나왔습니다.



업무 목록이 **23.6MB**였습니다. 조직 전체 범위에서 업무 2,100건에 주차 스냅샷 109,200개가 실려 나왔고, 그중 19MB는 화면이 한 번도 그리지 않는 주차별 본문이었습니다. 목록 표는 제목·진척도·경과·상태만 읽고, 주차별 기록은 상세 대화상자를 열었을 때만 쓰입니다.

- 목록에서 `weeks` 를 뺐습니다. 상세는 `GET /api/v1/work-items/{id}` 로 한 건만 가져옵니다.
- 상한과 전체 건수를 함께 냅니다. v0.25에서 세운 규칙("목록을 반환하는 API는 상한과 전체 건수를 함께 갖습니다")을 이 경로만 지키지 않고 있었습니다. 화면도 "2,100건 중 200건"이라고 적습니다.
- 실측: **23,603,838B → 164,591B (143배)**. 상세 1건은 10,980B입니다.

나머지는 측정만 하고 남겨 둡니다. 경영 요약 1,183ms, 기간 보고(연간) 1,334ms, 업무 그래프 1,280ms입니다. 원인은 같습니다 — `loadWorkItems` 가 109,200행을 전부 읽어 요약한 뒤 대부분을 버립니다. 고치려면 집계를 SQL로 내려야 하고, 그것은 이 회차의 범위를 넘습니다. 응답 크기와 달리 지연은 300명 규모에서 아직 견딜 만한 수준이라 순서를 뒤로 둡니다.

백필은 문제가 아니었습니다. 초기 로그의 속도(500건/3.7초)로 109,200건이면 13분으로 보였지만, 그것은 업무 2,100건을 새로 만드는 구간의 비용이었고 그 뒤로는 500건/0.25초로 올라가 전체 82초에 끝났습니다. 다만 진행 로그의 `pending` 이 첫 배치의 총계에서 움직이지 않아, 지켜보는 사람에게는 남은 양이 얼어붙은 것처럼 보입니다.

#### 3.10.6 v0.53.0에서 완료 — 새로고침마다 달라지던 협업 목록

v0.52에서 "지연은 나중에"라고 남긴 것을 재러 갔다가, 지연보다 큰 것을 먼저 찾았습니다.

같은 데이터·같은 바이너리로 업무 그래프를 두 번 부르면 협업 목록이 달라졌습니다. 팀 10의 짝이 한 번은 팀 9, 다음 번은 팀 2로 나왔습니다. 원인은 정렬의 동점 처리입니다. 행은 `map`에서 나오므로 도착 순서가 매번 다른데, 정렬 기준이 공유 업무 수와 왼쪽 조직명뿐이라 한 조직의 모든 짝이 동점이었고, `SliceStable` 이 그 임의의 도착 순서를 그대로 보존했습니다. 오른쪽 조직명을 마지막 기준으로 넣어 순서를 전순서로 만들었습니다.

측정 방법이 틀렸던 것도 정정합니다. v0.52에서 기록한 경영 요약 1,183ms는 제가 만든 시드가 모든 제목을 "업무 N 처리"로 찍어 낸 탓이었습니다. 모든 쌍이 토큰을 공유하니 `linkWorkItems` 의 2.2백만 번 비교가 전부 끝까지 갔습니다. 어휘를 흘뜨린 현실적인 제목으로 다시 재니 같은 규모에서 175ms입니다.

그 위에 최적화를 하나 없었습니다. 의미 있는 단어를 하나도 공유하지 않는 쌍은 어차피 버려지는데 유사도를 먼저 계산하고 있었습니다. 단어 역색인으로 살아남을 수 있는 쌍만 비교합니다. **결과는 동일하고**(응답 바이트 단위로 대조), 업무 그래프 175-190ms → 121-154ms, 경영 요약 172-187ms → 101-115ms입니다.

#### 3.10.7 v0.54.0에서 완료 — 결정성 전면 점검과 기간 보고 응답

v0.53에서 고친 비결정성이 한 곳뿐인지 확인할 근거가 없어서, 눈에 띄기를 기다리지 않고 기계적으로 훑었습니다.

**결정성 점검은 깨끗했습니다.** 300명·109,200건 인스턴스에서 주요 GET 28개를 각각 5회씩 불러 바이트 단위로 대조했습니다. 관리자와 팀장 두 역할로, 결정 700건과 선행·후행 링크 419건을 동점이 많이 생기도록 심은 뒤에도 어긋난 곳은 없었습니다. 기간 보고 세 개만 달랐는데 차이는 `generatedAt` 하나였습니다. v0.53의 `edges()` 가 유일한 사례였습니다.

**대신 훑는 중에 v0.52와 같은 구조를 하나 더 찾았습니다.** 연간 조직 전체 기간 보고가 885KB인데 그중 522KB가 업무별 주차 계열이었습니다. 타임라인은 상위 20건만 그리고 아래 표는 주차 계열을 아예 읽지 않습니다. 나머지 236건의 주차별 본문은 내려받아 버려집니다.

- 앞의 `timelineItems` 건만 주차 계열을 싹습니다. 몇 건인지는 서버가 정해서 응답에 담습니다 — 화면이 숫자를 따로 갖고 있으면 둘이 어긋나는 순간 빈 행을 그리게 됩니다.
- 항목 자체는 전부 보냅니다. 표는 그것을 다 씁니다.
- CSV·PPTX는 별도 경로이고 전체 뷰에서 만들므로 영향이 없습니다. CSV 257줄(헤더 포함 256건)을 확인했습니다.
- 실측: 연간 807KB → 332KB, 분기 512KB → 278KB, 월간 → 266KB.

#### 3.10.8 v0.55.0에서 완료 — 규모 점검을 명령 한 줄로

v0.52·v0.53·v0.54의 발견은 셋 다 큰 데이터에서만 드러났습니다. 그런데 그 방법이 제 손안에만 있었습니다. 매번 시드 SQL을 새로 쓰고 훑는 스크립트를 새로 짜는 한, 다음 기능이 같은 함정을 다시 만들어도 아무도 모릅니다.

- `scripts/seed-scale.sql` — 조직 41개·사용자 300명·52주·보고 항목 109,200건. 빈 데이터베이스에만 실행되도록 막아 두었습니다.
- `scripts/scale-check.py` — 주요 조회 경로를 5회씩 불러 크기·지연·결정성·상한 표기를 봅니다. 지적 사항이 있으면 종료 코드 1입니다.

만들자마자 두 가지를 잡았습니다.

**회의 모드의 변경점 섹션이 2,100건이었습니다.** 상한도 전체 건수도 없이 전 조직 업무를 그대로 싹였습니다. "회의에서 다뤄야 할 것만" 보여 준다는 화면이 전 조직 목록을 안건이라고 내놓고 있었습니다. 섹션마다 40건 상한과 전체 건수를 넣고, 잘라 낼 때 **덜 중요한 것이 잘리도록** 정렬했습니다 — 진척이 뒤로 간 업무가 먼저, 그다음 변화 폭이 큰 순서입니다. 예전에는 도착 순, 즉 업무 식별자가 작은 순이었는데 그건 논의할 이유가 아닙니다. 회의 응답 744,820B → 21,291B.

업무 인사이트의 반복 업무 목록이 **934건**이었습니다. 같은 응답 안의 유사 업무·중복 의심은 상한과 전체 건수를 갖고 있는데 이것만 없었습니다. 같은 상한(200)과 전체 건수를 붙였습니다.

체크 자체도 한 번 고쳤습니다. 처음에는 `duplicates` 의 개수를 `duplicateTotal` (단수)로 세는 것을 못 알아보고 다섯 번 연속 오탐을 냈습니다. 우는 늑대가 되는 체크는 읽히지 않게 되므로, 개수 이름의 여러 표기를 인정하고 한 단계 아래 중첩된 개수도 찾도록 고쳤습니다.

시드도 두 번 고쳤습니다. 처음에는 마지막 주 진척이 전부 100%에 닿아 "지난주 대비 변화"가 0건이 됐고, 그다음에는 PostgreSQL이 UTC라 일요일 저녁에 시드의 마지막 주가 애플리케이션이 보는 주보다 한 주 앞이었습니다. 후자는 보고 누락 **1,866건** · 변경점 **0건**으로 나타나 변화 감지 버그처럼 보였지만 시드 버그였습니다.

남은 것(체크가 지적한 **5건**): 기간 보고의 `contributors` (300건), `/team/members`, `/admin/users` 가 전체 건수 없이 전량을 반환합니다. 잘리지는 않지만 사용자 1만 명이면 메가바이트 단위가 됩니다.

#### 3.10.9 v0.56.0에서 완료 — 규모 점검 지적 사항 0건

v0.55의 체크가 남긴 5건을 처리했습니다. 처리하면서 셋 다 성격이 달랐다는 것이 드러났습니다.

`/admin/users` 는 진짜 문제였습니다. 전량을 반환했고(300명에 63KB), 화면에는 검색이 아예 없어서 특정 계정을 찾는 방법이 브라우저의 페이지 내 찾기뿐이었습니다. 1만 명이면 메가바이트입니다.

- 상한(기본 100)과 전체 건수를 넣되 **검색을 함께** 넣었습니다. 찾을 방법 없는 상한은 무제한보다 나쁩니다 — 필요한 계정이 그냥 사라지기 때문입니다. `q` 는 아이디·표시 이름·이메일을 함께 봅니다.
- 검토 책임자 선택기가 표의 목록을 재사용하고 있었습니다. 상한을 걸면 조직이 커질수록 검토자가 조용히 빠지고, 보는 사람은 그 이름이 자격이 없어서 없는 건지 페이지 밖이라서 없는 건지 알 수 없습니다. 역할 필터(`role=`)로 따로 불러오게 했습니다. 검토자는 조직이 아무리 커도 작은 집합입니다.
- 실측: 63,354B → 21,195B. 검색 `u100` → 1건, 역할 필터 → 13명.

나머지 **4건**은 위반이 아니었습니다. 기간 보고의 `contributors` 와 `/team/members` 는 전량을 반환합니다 — 잘리지 않으니 "일부를 전부처럼 보여 준다"는 규칙을 어기지 않습니다.

그래서 체크가 **추측 대신 확인**하게 고쳤습니다. 개수가 없는 큰 목록을 만나면 같은 경로를 `limit=1` 로 한 번 더 불러 봅니다. 응답이 바이트 단위로 같으면 그 경로는 페이징을 하지 않는다는 뜻이고, 전량 반환이므로 완전합니다. 짧아지면 말없이 페이징하고 있다는 뜻이고 그것이 진짜 위반입니다. 전자는 "상한 없이 전량 반환"으로 따로 보고하되 종료 코드에 반영하지 않습니다.

이제 `scripts/scale-check.py` 가 **22개 경로, 지적 사항 0건**으로 끝납니다.

#### 3.10.10 v0.57.0에서 발견 — 월요일 아침 로그인에 컨테이너를 죽입니다

지금까지의 측정은 전부 **요청 하나씩**이었습니다. 응답 크기도 질의 계획도 건강한데, 실제로 배포를 무너뜨리는 실패는 거기 없었습니다.

300명 규모에 동시 300 세션(쓰기 250·읽기 50)을 걸자 컨테이너가 **exit 137, OOM**으로 죽었습니다. 최대 메모리 12GB였습니다.

원인을 짚기 전에 제 귀인이 한 번 틀렸습니다. 경로별로 메모리를 재니 네 경로 모두 30 동시에 2GB였고 응답이 48KB인 경로까지 그랬습니다. 데이터 크기로 설명되지 않아 힙 프로파일을 떼더니 **98%가 argon2.initBlocks**, 즉 로그인이었습니다. 조회 경로가 아니라 부하 스크립트가 매 스레드에서 로그인하고 있었던 것입니다.

Argon2id는 검증 한 번에 64MiB를 잡고 그 시간 동안 붙들고 있습니다. 로그인이 하나씩 올 때는 아무도 눈치채지 못합니다. **월요일 아홉 시에 250명이 동시에 들어오면 그들끼리 16GB를 잡습니다** — 그 주의 첫 요청이, 그 주에서 가장 바쁜 1분에.

- 검증과 해싱을 큐에 넣었습니다. 동시 실행은 코어 수를 따르되 **최소 2, 최대 8**입니다. 검증 1회가 61ms 이므로 조직 전체가 몇 초 안에 로그인하고, 최대 메모리는 무엇이 오든  $8 \times 64\text{MiB}$ 로 묶입니다.
- 상한이 8인 이유는 그 이상이 처리량을 사지 못하기 때문입니다(CPU·메모리 바운드이지 대기가 아닙니다). 반면 슬롯 하나는 컨테이너가 갖고 있어야 할 64MiB입니다.
- 비용을 낮추는 선택은 하지 않았습니다.** 그것은 스케줄링 문제를 풀자고 저장된 모든 비밀번호를 약하게 만드는 일입니다.

실측(동시 250 쓰기·50 읽기·4회전):

|        | 이전        | 이후                                                |
|--------|-----------|---------------------------------------------------|
| 최대 메모리 | 12.01 GiB | 1.14 GiB (실사용 힙 512MB = $8 \times 64\text{MiB}$ ) |
| 저장 중앙값 | 264ms     | 18ms                                              |
| 저장 p99 | 390ms     | 51ms                                              |
| 조회 p99 | 782ms     | 204ms                                             |
| 결과     | OOM 종료    | 오류 0건                                             |

`scripts/load-check.py` 로 남겼습니다. 다음에 같은 실패가 생기면 손으로 재현하지 않아도 됩니다.

#### 3.10.11 v0.58.0에서 완료 — 방금 만든 상한이 배포 한도를 통째로 먹고 있었습니다

v0.57에서 argon2 동시 실행을 8개로 묶었습니다. 그리고 나서 배포 매니페스트를 열어 보니 메모리 한도가 **512Mi**였습니다.  $8 \times 64\text{MiB} = 512\text{MiB}$ . 상한이 한도 전부입니다.

매니페스트 그대로(cpu 1, memory 512Mi) 재현하니 v0.57 수정이 있어도 `OOMKilled=true` 로 죽었습니다.

원인은 둘이었습니다.

**첫째, 풀 크기가 컨테이너를 모릅니다.** `runtime.NumCPU()` 는 호스트의 코어 수를 봅니다. 32코어 노드에서 CPU 1개짜리 파드도 슬롯 8개를 열었습니다. OOM을 막으려고 만든 상한이 스스로 OOM을 냈습니다. 이제 cgroup v2의 `memory.max` 를 읽어 **감당 가능한 만큼만** 엽니다. 메모리가 구속 조건이므로 메모리가 결정하고, CPU는 그보다 더 줄일 때만 관여합니다.

**둘째, Go에게 한도를 알려 주지 않았습니다.** 5슬롯 320MiB로 줄여도 여전히 죽었습니다. 런타임은 30GB가 남은 기계를 보고 회수를 미루므로, 다 쓴 argon2 블록이 쌓여 RSS가 실사용의 두 배에 달합니다. `debug.SetMemoryLimit` 으로 컨테이너 한도의 7/8을 알려 주자 멈췄습니다.

기동 시 `password hashing pool sized` 로 슬롯 수·예약량·읽은 한도를 남깁니다. 운영자가 한도를 조정할 때 볼 숫자가 그것뿐이기 때문입니다.

실측(300명·109,200건, 동시 300 세션):

| 한도                   | 슬롯         | 결과                   |
|----------------------|------------|----------------------|
| 512Mi / 1 CPU (수정 전) | 8 (호스트 기준) | <b>OOM 종료</b>        |
| 512Mi / 1 CPU (수정 후) | 5          | 오류 0건, 48.8초, 최대 86% |
| 1Gi / 2 CPU (수정 후)   | 8          | 오류 0건, 31.3초         |

매니페스트 기본값을 1Gi / 2 CPU로 올리고 근거 숫자를 주석에 적었습니다. 512Mi도 동작하며 로그인도 느려질 뿐입니다 — 큐가 그것을 장애가 아니라 지연으로 만들어 줍니다.

#### 3.10.12 v0.59.0에서 완료 — 정상 동작이 로그를 채우고 있었습니다

v0.57 부하 시험 중에 눈에 걸렸지만 지나쳤던 둘을 봤습니다. 하나는 진짜였고 하나는 제 짐작이 틀렸습니다.

로그 폭주는 진짜였습니다. 부하 3.6초 동안 `report list truncated` 가 60줄 찍혔고, 그 시간의 유일한 로그가 그것이었습니다. 그때 실제로 무언가 잘못됐다면 "모든 것이 정상"이라는 보고 속에 묻혔을 겁니다.

상한이 목록을 자르는 것은 **사건이 아니라 상태**입니다. 이 배포가 누군가 정한 한도보다 크다는 뜻이고, 그것은 요청마다 참입니다. 요청마다 쓰는 것은 상태를 사건처럼 기록하는 일입니다.

- `conditionLog` 을 만들어 조건마다 **프로세스당 한 번만** 기록합니다. 재시작하면 다시 말하는데, 그때는 운영자가 어차피 보고 있습니다.
- 숫자를 잃지 않습니다. 잘린 응답은 이미 `total` 과 `limit` 을 담고 있고, 요청 지표 테이블이 호출 수를 셉니다.
- 같은 유형이던 `work graph truncated` 도 함께 옮겼습니다.
- 실측: 같은 부하에서 **60줄 → 1줄**.

지표 행 경합은 제 짐작이 틀렸습니다. 요청마다 같은 (`시각·메서드·경로·상태`) 행을 upsert하므로 줄을 셀 것이라 의심했는데, 재 보니 그 행 하나가 16-way 동시에서 **초당 9,411건**을 처리합니다. 앱의 최대치는 초당 38건이었습니다. 250배 여유입니다. 부하 중 `pg_stat_activity` 표본 60개에서 락 대기는 1건, 최대 동시 1이었습니다.

고칠 것이 없으므로 고치지 않았습니다. 숫자를 남기는 이유는 다음에 같은 의심이 들 때 다시 재지 않아도 되게 하기 위해서입니다.

#### 3.10.13 v0.60.0에서 완료 — 마감일 없이 기간 말을 묻습니다

v0.48~v0.51은 네 회차에 걸쳐 완료 시점 추정, 마감일 필드, 회의 합의 기한 연결, 경영 요약 반영을 만들었습니다. 전부 하나의 질문에 답합니다 — 제때 끝나는가. 그리고 전부 `work_items.due_date` 에 걸려 있는데, 실측이 **8건 중 0건**이었습니다. 아무도 입력할 이유가 없는 칸 하나에 네 회차가 묶여 있었습니다.

그런데 기간 보고는 이미 그 질문을 하고 있습니다. 분기 보고를 여는 사람은 "이번 분기에 뭐가 끝나나"를 묻고 있고, 그 경계는 사람이 아니라 달력이 정해서 요청 안에 이미 들어 있습니다. 같은 산술을 그 경계에 대고 돌리면 아무도 아무것도 입력하지 않아도 됩니다.

- 항목마다 `periodOutlook` — 두 속도로 기간 말 진척을 각각 계산해 구간으로 냅니다.
- **약속이 아니며 그런 척하지 않습니다.** 마감일은 누군가 약속했다는 뜻이고 기간 말은 보고가 여기서 끊긴다는 뜻입니다. 문구를 갈라 놓았고, 마감일이 있는 업무는 자기 전망을 따로 계속 갖습니다.
- 이미 끝난 기간에는 **추정하지 않습니다.** 결과가 이미 있는데 내놓는 추정은 예보가 아니라 예보처럼 보이는 잡음입니다.
- 기간 업무보고에 `기간 내 미완` 필터와 경영 인사이트 항목이 붙습니다.

만들면서 두 번 고쳤습니다. 첫 판은 263건 중 **217건**을 지적했는데 그건 발견이 아니라 목록 전체입니다. 원인은 진척이 늘지 않은 업무까지 추정한 것이었고 — 25%에 서 있는 업무에 "기간 말 15%"라고 답했습니다. 늘릴 속도가 없으면 추정하지 않고 정체 규칙에 맡깁니다. 그리고 완료 시점 자체가 없는 업무는 이미 **완료 시**

**점 불명** 이 보고하므로 이 제목에서 뺐습니다 — 같은 업무를 두 제목 아래 두 번 적는 것이 위험 목록을 안 읽히게 만드는 방법입니다.

실측: 13주에 걸쳐 주당 2.5%로 움직인 업무에 대해 마감일 입력 없이 "**이 속도로는 기간 말(2026-09-30)에 52%**". 종료된 분기(2026-Q1)에서는 256건 전부 침묵합니다.

#### 3.10.14 v0.61.0에서 완료 — 장애물이 어떻게 끝났는지 기록합니다

이슈는 주간보고에 나타났다가 몇 주 뒤 사라집니다. 지금까지 제품이 아는 것은 **중간**뿐입니다 — 이슈 3주 지속, 경영 요약에서 점수. 끝은 아무도 기록하지 않습니다. 그래서 **해결된 것과 포기한 것이 데이터에서 똑같아 보입니다**.

중간은 이미 주차별 행에서 유도할 수 있으니 여기서 다시 만들지 않습니다. 끝만 사람이 필요합니다. 이슈란 이 비었다는 것이 문제가 사라졌다는 뜻인지 적기를 그만뒀다는 뜻인지는 그 주에 작성한 사람만 압니다.

- 마이그레이션 018 `work_item_issue_outcomes` . 끝난 주·시작한 주·이어진 주 수·해소 문구를 담습니다.
- 편집기가 **그 한 주에 한 번** 묻습니다. 지난주 이슈가 있고 이번 주가 비었을 때만, 그 항목에만. **해소됨** 을 고르면 저장 시 기록하고, **아직 남음** 을 고르면 지난주 문구를 이번 주 이슈란에 되돌려 넣습니다 — 후자는 조용히 빠진 장애물을 되살리는 교정이라 그 자체로 값이 있습니다.
- 이어진 주 수는 **연속된 주만** 셉니다. 3월에 막혔다가 조용해지고 8월에 다른 것에 막힌 업무는 다섯 달짜리 장애물 하나가 아니라 두 개입니다. 공백을 건너뛰면 이 표 위에 세우는 모든 숫자가 부풀려집니다.
- 같은 주를 두 번 저장해도 한 번만 기록합니다. 보고서를 다시 저장하는 일은 일상이고, 그때마다 해소가 하나씩 늘면 안 됩니다.
- 기간 보고에 **이슈 해소** 지표 — 건수와 중앙값·최장. 평균이 아니라 중앙값인 이유는 1년 끌던 장애물 하나가 평균을 실제 장애물이 서 있지 않은 곳으로 끌고 가기 때문입니다.
- 아무도 답한 적이 없으면 0이 아니라 **-** 로 적습니다. 0은 "아무것도 시간이 안 걸린다"로 읽히고 그건 빈 표가 뜻하는 것의 반대입니다.

실측: 1주짜리와 4주짜리 에피소드를 각각 만들어 정확히 1과 4로 기록되는 것을, 그리고 4주짜리에서 그 앞 공백 이전의 이슈는 세지 않는 것을 확인했습니다. 기간 보고는 **2건 해소 · 중앙값 1주 · 최장 4주** 를 표시합니다.

이 표는 3.10.15(조직을 가로지르는 공유 장애물)의 재료이기도 합니다. 같은 장애물을 묶을 수 있게 되면 "이 장애물은 보통 N주 걸립니다"를 붙일 수 있습니다.

#### 3.10.15 v0.62.0에서 완료 — 막혔다고 쓰는 자리에서 무엇에 막혔는지 말하게



"조직을 가로지르는 공유 장애물"을 만들려고 이슈 텍스트 군집화를 제안했었습니다. 먼저 재 봤고, 만들지 않았습니 다. 군집을 평가할 실제 이슈 말뭉치가 어디에도 없습니다 — 시드는 2종(하나가 15,351회), 데모 데이터는 3종 12행입니다. 모델 품질을 젤 수 없는 기능을 만드는 것은 이 프로젝트가 Slack 알림과 문장 단위 근거에 대해 거부했던 것과 같은 일입니다.

대신 재는 중에 훨씬 확실한 것을 찾았습니다. 이슈 **12건**, 선언된 선행 링크 **0건**.

v0.40~42에서 선행·후행 링크, 병목 분석, 회의 안건의 **타 조직 대기**, 품질 점검의 차단 사유를 만들었습니다. 전부 링크를 읽습니다. 그리고 링크는 **한 건도 만들어지지 않았습니 다**. 마감일이 v0.60 전까지 그랬던 것과 정확히 같은 상태 — 완성돼 있고 구조적으로 도달 불가능합니다.

이유는 둘이었습니다.

- 선언하는 자리가 막혔다고 쓰는 자리와 다른 화면입니다. 이슈는 주간보고 편집기에 쓰고, 선행 관계는 업무 추적 상세에서 등록합니다. 이슈를 쓰는 곳에 선언을 가져왔습니다. 이미 등록한 사람에게는 등록된 내용을 보여 주고 다시 묻지 않습니다.
- 일반 사용자는 남의 업무를 찾을 수 없었습니다. 패널이 쓰던 `/work-items/search` 는 기본이 SELF 범위이고 팀장 미만에게는 조직 범위를 거부합니다. 선행 관계란 본디 남의 업무를 가리키는 것인데, 가리킬 대상을 찾는 길이 정작 막힌 사람 거의 전부에게 닫혀 있었습니다.

`/work-items/lookup` 을 새로 냈습니다. 배포 전체를 보되 선언이 어차피 화면에 표시할 네 가지만 돌려줍니다(식별자·제목·담당자·조직). 기존 검색이 싹는 이슈·해결 본문은 담지 않습니다. 노출이 넓어지는 변경이므로 코드와 문서에 그대로 적어 두었습니다.

실측: 일반 사용자(팀 6)가 편집기에서 다른 조직(팀 9)의 업무를 선행으로 등록했고, 그것이 팀장의 회의 자료 **타 조직 대기** 에 나타나는 것까지 확인했습니다. 2글자 질의도 답합니다.

#### 3.10.16 v0.63.0에서 완료 — 주간보고 제품에 주간 취합이 없었습니다

"팀장 보고서를 팀원 보고서에서 AI로 초안 생성"을 제안했었습니다. 만들기 전에 켜고, 전제가 틀렸습니다.

데모 데이터에서 팀장(choi)의 주간보고와 팀원 보고서의 겹침이 0입니다. 팀장은 팀원 것을 요약하지 않고 자기 업무("조직 KPI 정리")를 씁니다. 이 제품의 모델에서 팀 취합은 개인 보고서가 아니라 별도 화면의 일입니다.

그래서 진짜 빈 곳을 찾았습니다. `PeriodKind = MONTH | QUARTER | HALF | YEAR` — 주간이 없습니다. 팀 주간보고 화면은 팀원 보고서를 한 건씩 나열하고, 중복을 제거해 한 장으로 만드는 취합은 월간부터 시작합니다. 주간보고 제품에서 주 단위 팀 취합이 빠져 있었습니다.

- `kind=WEEK` 를 더했습니다. 식별자는 그 주의 시작일( `yyyy-mm-dd` )로, `weekly_reports.week_start` 와 같은 형식입니다. ISO 주차 번호를 새로 만들면 한 주를 부르는 방식이 둘이 되고 시작 요일을 바꾸는

순간 어긋납니다.

- 시작일이 아닌 날짜를 주면 그 주로 맞춥니다. 수요일을 넘긴 호출자는 그 주를 뜻하지, 수요일부터 화요일 까지를 뜻하지 않습니다.
- 주 시작 요일은 배포 설정이므로 경계도 그것을 따릅니다.
- 아래쪽은 전부 그대로 동작합니다 — 중복 제거, 인사이트, 하이라이트, CSV·PPTX 내보내기, 발표 모드.

실측: 팀장이 팀원 10명의 주간보고를 **업무 21건으로 정리한 한 장**으로 봅니다. CSV 56건, PPTX 15슬라이드.

#### 3.10.17 v0.63.0에서 함께 고침 — UTC로 하루씩 밀리던 날짜 네 곳

주간 선택기를 브라우저에서 확인하다가 잡았습니다. 선택지는 2026-08-23 (이번 주) 인데 서버는 2026-08-17 주차 로 답했습니다. `Date.toISOString()` 이 UTC로 답하므로, KST 자정은 전날이 됩니다.

같은 실수가 네 곳에 있었고 세 곳은 이미 배포돼 있었습니다.

- 회의 모드의 주 이동이 하루씩 어긋났습니다. 08-24에서 다음 주 ▶ 를 누르면 08-31이 아니라 **08-30** — 어느 주의 시작도 아닌 날짜이고, 그 주 안건은 아무것도 찾지 못합니다.
- 결정 기록 화면이 오전 9시 이전에 기록하는 사람에게 어제 날짜를 기본값으로 내놓았습니다.
- 주간 선택기가 지난주를 "이번 주"라고 적었습니다.
- 타임라인 차트의 주차 키.

`localdate.ts` 하나로 모았습니다. 브라우저가 이미 지역 시간으로 갖고 있는 값을 읽지, 다른 시간대로 바꿔 문자열을 자르지 않습니다. Go 가드 테스트가 `toISOString().slice(0,10)` 의 재등장을 막습니다 — 이 실수는 호출부에서 완벽히 합리적으로 보이고, 아무도 테스트하지 않는 시간대에서만 드러납니다.

#### 3.10.18 v0.64.0에서 완료 — 만들어 두고 부르지 않은 것을 기계로 찾습니다

세 회차 연속 같은 모양이었습니다. 완성돼 있고, 테스트도 문서도 있고, 아무 화면도 부르지 않아 쓸 수 없는 기능. 마감일(v0.60), 선행 링크(v0.62), 주간 취합(v0.63). 셋 다 코드를 읽어서가 아니라 실행 중인 배포를 재다가 빈 표를 보고 찾았습니다.

그래서 싸게 훑었습니다. 서버가 등록한 96개 경로를 프론트엔드 소스와 대조했습니다. 진짜 후보는 셋이었고, 그중 하나가 실물이었습니다.

보고서 코멘트가 한쪽으로만 흐르고 있었습니다. 검토자가 할 수 있는 일은 승인 또는 반려 둘뿐이라, 질문 하나를 하려면 반려해야 했습니다 — 상태를 바꾸고 남의 책상으로 일을 돌려보내는 일입니다. 대부분은 그래서 아무 말도 하지 않습니다. 작성자 화면은 검토 의견을 보여주지만 했고, 그 칸은 반려로만 채워질 수 있었으며, 답할 방법이 없었습니다.

`POST /reports/{id}/comments` 는 권한 검사·길이 제한·감사 기록까지 갖춘 채 아무도 부르지 않고 있었습니다. 양쪽에 자리를 만들었습니다. **보고서 상태는 바뀌지 않습니다.**

나머지 둘은 이유를 적었습니다 — 후보 출처는 목록 응답이 이미 담고 있고, 미매핑 사용자는 `users/mappings` 가 전체 사용자를 LEFT JOIN으로 담아 매핑 제안까지 주는 것의 필터판입니다.

가드를 남기면서 한 번 고쳤습니다. 첫 판은 접미사를 느슨하게 찾아, CSS 클래스 `"comments"` 가 검사를 통과시켰습니다. 코멘트 호출을 지워도 테스트가 통과했습니다 — **짓지 못하는 감시견은 없느니만 못합니다. 믿게 되니까요.** URL 경로 조각( `/comments` )으로만 인정하게 좁히고, 경로를 `${id}/${action}` 으로 조립하는 승인·반려는 이유를 적어 예외로 두었습니다.

#### 3.10.19 v0.65.0에서 완료 — 반대 방향으로도 훑었습니다

v0.64가 "서버가 가진 것 중 화면이 안 부르는 경로"를 찾았습니다. 반대도 같은 값이 있습니다 — 데이터베이스와 응답이 가진 것 중 아무도 읽지 않는 것.

컬럼 쪽은 깨끗했습니다. 284개 중 Go 코드가 언급하지 않는 것은 둘뿐이고 ( `schema_migrations.applied_at` 은 기본값으로만 쓰이는 기록용, `work_items.parent_work_item_id` 는 쓰이지 않는 계층 구조 자리), 전부 NULL인 컬럼 14개는 모두 데모가 결재 흐름을 거치지 않은 결과로 설명됩니다. 특히 `report_status_history.from_status` 가 35행 전부 NULL이라 의심했지만, 데모에 초안 생성만 있고 **생성에는 이전 상태가 없는 것이 맞습니다.**

응답 필드 쪽에서 하나 나왔습니다. 371개 중 화면이 이름조차 언급하지 않는 것이 20개인데, 19개는 남의 전문입니다(Confluence·OpenAI·JSON-RPC·OIDC). 우리 것은 하나 — `movedItems` .

병합·분리 API는 실제로 옮겨진 주차 기록 수를 돌려주고, 서버 주석은 그것을 "정정이 의도대로 됐는지 알려주는 유일한 숫자"라고 적어 두었습니다. 화면은 그것을 버리고 **요청한 수**를 알렸습니다. 합치기는 아예 수를 말하지 않았습니다.

이제 실제 수를 말합니다. 분리는 요청 수와 다를 때 다르게 적고(서버가 어긋난 id를 400으로 막으므로 지금은 방어적입니다), 합치기는 옮겨진 기록 수를 알립니다 — 화면이 전에는 알 수 없던 숫자입니다.

가드를 하나 더 남겼습니다. 응답에 실려 나가는데 화면이 이름조차 언급하지 않는 필드가 생기면 실패합니다. 남의 전문은 파일 단위로 제외하고, 정당한 예외는 이유를 적어야 통과합니다. `movedItems` 를 다시 버리게 만들어 실제로 실패하는 것을 확인했습니다.

#### 3.10.20 v0.66.0에서 완료 — 데이터베이스가 끊겼을 때 화면이 하던 거짓말

지금까지 잔 것은 전부 정상 흐름이었습니다 — 크기, 지연, 동시성, 도달 가능성. 오프라인망 배포에서 실제로 사람을 곤란하게 만드는 것은 보통 **잘못됐을 때 화면이 뭐라고 하는가**입니다. 그래서 PostgreSQL 연결을 실제로 끊고 봤습니다.

가장 중요한 것은 이미 났습니다. 보고서를 쓰던 중에 데이터베이스가 사라져도 작성 중인 내용이 남고, 복구 후 그대로 저장됩니다.

그런데 화면이 두 번 거짓말했습니다.

- 저장을 누르면 "로그인이 필요합니다" 라고 했습니다. 세션 조회가 데이터베이스를 거치는데, "세션이 없다"와 "세션을 확인할 수 없다"를 같은 오류로 돌려주고 있었습니다. 게다가 화면은 401을 세션 만료로 다루므로 새 탭에서 로그인하라고 권했고, 그것을 따르면 쓰던 보고서가 사라집니다.
- 로그인을 누르면 "아이디 또는 비밀번호가 올바르지 않습니다" 라고 했습니다. 사용자 조회의 오류가 자격 증명 실패 분기로 흘러들었기 때문입니다. 더 나쁜 것은 그 시도가 로그인 차단 카운터에 기록된다는 점입니다 — 가장 열심히 재시도한 사람이 잠깁니다. 한 번도 그들을 거부한 적 없는 서비스에서.

둘 다 원인이 같습니다. 모른다는 것과 아니라는 것을 구분하지 않았습니다. `errNoSession` 표지를 두어 진짜로 로그인하지 않은 경우에만 401을 주고, 나머지는 503과 "로그인은 유효하고 작성 중인 내용도 남아 있으니 잠시 후 다시"를 말합니다.

그리고 매달렸습니다. 로그인 화면은 20초, 로그인 요청은 25초 넘게 응답이 없었습니다. 원인은 곱셈입니다 — 실패하는 질의 하나가 연결 시도 시간을 다 쓰는데 핸들러는 여러 번 질의합니다(로그인 화면만 설정 4개). 각 호출에 기한을 붙였습니다. 이 패턴은 이미 `readyz` 가 자기 질의에 쓰고 있었고, 다른 경로에만 없었습니다.

|           | 이전                      | 이후                   |
|-----------|-------------------------|----------------------|
| 로그인 화면    | 20초 매달림                 | 5.6초 · 200(기본값으로 렌더) |
| 로그인 요청    | 25초 매달림                 | 12초 · 503 · 정확한 원인   |
| 저장 중 장애   | "로그인이 필요합니다" + 세션 만료 배너 | 원인을 말하고, 배너 없음       |
| 작성 중이던 내용 | 유지됨                     | 유지됨                  |

12초는 여전히 깁니다. 설정·차단 카운터·사용자 조회의 기한이 순차로 더해진 값이며, 한 번 실패한 뒤 잠시 즉시 실패시키는 장치를 두면 줄어듭니다. 지금은 무한정 매달리지 않고 정확한 답을 준다는 것까지가 확인된 범위입니다.

#### 3.10.21 v0.67.0에서 완료 — 실패한 업로드가 디스크를 영구히 먹고 있었습니다

v0.66이 데이터베이스를 끊었으니 이번엔 디스크를 채웠습니다. 320KB짜리 볼륨에 이미지를 올려 봤습니다.

구조는 이미 옳았습니다. 업로드는 파일을 먼저 쓰고 행을 나중에 넣으므로, 쓰기가 실패하면 행이 생기지 않습니다. "목록에는 있는데 열면 404"가 되는 순서가 아닙니다. 백업 스크립트가 같은 이유로 그 순서를 지킨다고 적어 둔 것과 일치합니다.

그런데 실패한 쓰기가 조각 파일을 남겼습니다. `os.WriteFile` 은 대상 파일을 만들고 그 안에 씁니다. 디스크가 도중에 차면 잘린 파일이 최종 이름으로 남습니다. 행은 없으니 그 파일을 가리키는 것도 없고, 이 제품의 어떤 정리 경로도 그것을 찾지 못합니다 — 부팅 시 점검은 행에서 파일을 찾지, 반대로는 보지 않습니다.

실측: 실패 한 번이 **320KB 중 319KB**를 먹고 그대로 잠갔습니다. 이후 모든 업로드가 실패합니다.

고치는 방법은 이 코드베이스 안에 이미 있었습니다. PPTX 템플릿과 Import 저장소는 **임시 파일에 쓰고 이름을 바꾸며** 실패 시 `defer os.Remove` 로 치웁니다. 첨부만 맨 `os.WriteFile` 이었습니다. 같은 패턴으로 맞췄고, 덤으로 최종 이름이 절반만 쓰인 파일을 가리키는 순간이 사라집니다.

메시지도 고쳤습니다. "이미지를 저장할 수 없습니다"는 고칠 수 있는 사람(관리자)이 보지 않는 말이었습니다. 공간 부족일 때는 507과 함께 원인과 확인할 볼륨을 말합니다.

|          | 이전               | 이후         |
|----------|------------------|------------|
| 실패 후 디스크 | 100% 사용, 영구      | 1% 사용, 회복  |
| 남은 조각 파일 | 319KB            | 없음         |
| 응답       | 500 "저장할 수 없습니다" | 507 원인과 조치 |
| DB 행     | 생기지 않음           | 생기지 않음     |

#### 3.10.22 v0.68.0에서 완료 — 네 가지 고장이 한 문장으로 나오고 있었습니다

데이터베이스(v0.66)와 디스크(v0.67)를 끊어 봤으니, 실패 경로에서 남은 절반은 **AI 게이트웨이와 Confluence**였습니다. 응답하지 않는 주소, 401을 주는 서버, HTML을 돌려주는 엔드포인트를 실제로 붙여 놓고 화면이 뭐라고 하는지 읽었습니다.

**AI** 쪽은 네 가지 서로 다른 고장이 한 글자도 다르지 않은 문장으로 나왔습니다.

| 실제 고장             | 이전 화면 문구                                                     |
|-------------------|--------------------------------------------------------------|
| API Key가 거부됨(401) | AI Gateway가 유효한 구조화 결과를 반환하지 못했습니다. 관리자 설정과 모델 지원 여부를 확인하세요. |
| 게이트웨이 내부 오류(500)  | (완전히 같은 문장)                                                  |

프록시가 로그인 페이지를 돌려 (완전히 같은 문장)  
줌

모델이 Structured Output (완전히 같은 문장)  
미지원

넷 중 셋에 대해 그 문장은 틀린 곳을 가리킵니다. 키가 거부된 관리자는 모델 지원 여부를 확인하러 가고, 프록시가 가로챈 관리자도 같은 곳으로 갑니다. 마지막 한 줄만 우연히 맞습니다.

이제 일곱 가지 고장이 각각 다른 말을 하고, 각자 다음에 무엇을 할지를 말합니다. 401/403은 API Key를, 404는 Endpoint가 `/v1/chat/completions` 로 끝나는지를, 5xx는 "게이트웨이 쪽 문제이므로 이 설정을 바꿔도 해결되지 않습니다"를, 비-JSON 응답은 프록시나 로그인 페이지를, 연결 실패는 사내망 접근 경로를 가리킵니다.

**Confluence 쪽은 반대 문제였습니다** — 말을 안 한 게 아니라 내부를 그대로 뱉었습니다.

```
decode Confluence response: invalid character '<' looking for beginning of value
Get "http://confluence.internal:8090/rest/api/content/search?q=type+%3D+page+and+lastmod..."
```

설정 화면은 내부 주소와 질의문이 처음 등장할 자리가 아닙니다. 원문은 로그로 보내고, 화면에는 401이면 연동 계정을, 404면 Base URL이 Confluence 루트인지를, 인증서 오류면 사내 CA를 확인하라고 적습니다.

가드가 실제로 무는지 확인했습니다. `aiUserMessage` 를 원래의 한 문장으로 되돌리자 테스트가 실패했습니다. 그런데 Confluence 테스트는 원문 노출을 되돌려도 통과했습니다 — 사례가 전부 앞쪽 분기에 걸려 마지막 폴백을 건드리지 않았기 때문입니다. 알아보지 못한 오류야말로 아무도 문구를 읽어 본 적 없는 오류이므로, 그 경로를 덮는 사례를 더해 다시 확인했습니다. 짓지 못하는 감시건은 없느니만 못하다는 v0.64의 교훈이 이번에는 절반만 짓는 형태로 나타났습니다.

원칙으로 남깁니다. 서로 다른 원인은 서로 다른 문장으로 말합니다. 원인을 구분하지 못하는 하나의 문장은 넷 중 셋을 틀린 곳으로 보냅니다.

#### 3.10.23 v0.69.0에서 완료 — 나오지 않은 것은 아무도 세지 않았습니다

v0.68까지 확인한 AI 실패는 전부 답을 못 받는 실패였습니다. 남은 것은 반대쪽입니다. 게이트웨이가 200을 주고, 스키마에 맞는 JSON을 주고, 신뢰도까지 0.9를 붙여 놓았는데 내용이 모자란 경우.

10장짜리 덱(그중 2장은 빈 간지)을 올리고, 8장의 내용 중 3장만 인용하는 완벽하게 유효한 응답을 돌려주는 게이트웨이를 붙였습니다.

|        | v0.68.0      | v0.69.0              |
|--------|--------------|----------------------|
| 상태     | READY        | NEEDS_REVIEW         |
| 확정 화면  | 기본 선택됨       | 직접 선택해야 함            |
| 경고     | 0건           | 빠진 슬라이드 번호를 지목       |
| 사라진 업무 | 5장 분량, 흔적 없음 | 5, 6, 7, 9, 10번으로 명시 |

`validateAIResult` 는 이미 꼼꼼합니다. 인용된 슬라이드가 실제 입력에 있는지, 진행률이 범위 안인지, 같은 업무가 중복되지 않는지 봅니다. 그런데 검사 대상이 전부 나온 것입니다. `pptxSourceSlides(input)` 로 입력 슬라이드 전체 집합을 이미 계산해 두고도 반대 방향 — 아무도 인용하지 않은 슬라이드 — 은 세지 않았습니다.

빈 슬라이드는 세지 않습니다. 간지와 빈 표만 있는 슬라이드까지 지목하면 모든 Import가 경고를 달고, 그러면 아무도 경고를 읽지 않게 됩니다. 정규화 텍스트에서 `[SHAPE n]` · `[TABLE n]` · `[ROW n]` · `[COL n]` · `[EMPTY]` 같은 뼈대만 남은 슬라이드를 제외했습니다. 잘림 안내가 있는 슬라이드는 내용으로 셉니다 — 보낸 것보다 많았지 적지 않았습니다.

경고와 상태를 갈라 놓지 않았습니다. 처음엔 판정을 `validateAIResult` 안에 넣었는데, 그러면 경고는 뜨지만 파일은 READY로 남아 확정 화면에서 기본 선택됩니다. `finalizeImportedAIResult` 의 다른 모든 경고가 `NEEDS_REVIEW` 를 함께 세우는 것과 어긋납니다. 판정을 그쪽으로 옮겨 둘을 붙였습니다.

임계값은 지어내지 않았습니다. 실제 덱 말뭉치가 없으므로 "몇 퍼센트 이상 누락이면 검토"를 정할 근거가 없습니다(4.5의 원칙). 대신 이 코드베이스가 이미 정해 둔 기준을 씁니다 — 슬라이드 \*일부\*가 입력 한도로 잘린 것만으로도 `NEEDS_REVIEW` 가 됩니다. 슬라이드 \*전체\*가 보고되지 않은 것은 그것의 무거운 쪽이므로 더 낮은 대우를 받을 수 없습니다. 대가는 "감사합니다" 장으로 끝나는 덱이 필요 없는 클릭 한 번을 요구한다는 것이고, 실제 덱이 모이면 그것이 다름을 지점입니다.

원칙으로 남깁니다. 검증은 나온 것만 보면 절반입니다. 200과 유효한 JSON과 높은 신뢰도를 동시에 갖춘 응답도 틀릴 수 있고, 그 틀림은 화면에 없는 것의 모양으로 나타납니다. 사람은 있는 것을 검토하지 없는 것을 검토하지 못합니다.

#### 3.10.24 v0.70.0에서 완료 — 미매핑을 세고 있었지만 방향이 반대였습니다



v0.69에서 얻은 원칙 — 검증은 나온 것만 보면 절반이다 — 을 Confluence 동기화에 그대로 물었습니다. 답은 같았습니다.

가짜 Confluence를 붙여 12개 페이지를 내보냈습니다. 작성자는 j.hkjang , k.lee , p.park — Weekly 계정과 아이디가 다른 사람들입니다. 관리자 화면은 이렇게 나왔습니다.

|                      | v0.69.0 | v0.70.0    |
|----------------------|---------|------------|
| 동기화 상태               | SUCCESS | SUCCESS    |
| 조회 / 새 초안            | 12 / 0  | 12 / 0     |
| 실패                   | 0건      | 0건         |
| 매핑 / 미매핑             | 1 / 0   | 1 / 0      |
| 최근 진단                | 0건      | 아이디 3개를 지목 |
| 연결 안 된 Confluence 계정 | 없는 칸    | 3명         |
| 초안이 못 된 페이지          | 없는 칸    | 12개        |

미매핑 사용자 기능은 이미 있었습니다. 다만 세는 방향이 반대였습니다.

```
SELECT count(*) FROM users u WHERE u.active=true AND NOT EXISTS(
  SELECT 1 FROM user_external_accounts e WHERE e.user_id=u.id AND ... )
```

이것은 우리 쪽 사람 중 연결 안 된 사람을 셉니다. 그런데 업무가 사라지는 지점은 반대편입니다 — 스캔에 나타났는데 아무에게도 붙지 못한 **Confluence** 계정. 두 숫자는 동시에 성립합니다. Weekly 사용자가 전원 연결돼 미매핑 0명인 상태에서, 12개 페이지가 전부 버려질 수 있습니다. 실제로 그렇게 나왔습니다.

buildConfluenceActivities 는 mappings[...] 가 돌려준 0을 보고 creatorID > 0 검사에서 조용히 넘어갑니다. 실패가 아니므로 PagesFailed 도 오르지 않고, 만들어진 것이 없으므로 CandidatesCreated 도 그대로입니다. 네 개 카운터 어디에도 이 사건이 들어갈 칸이 없었습니다.

진단은 이름을 말합니다. 개수만으로는 고칠 수 없습니다 — 관리자가 매핑 화면에 입력해야 하는 것은 아이디 이기 때문입니다. 실제로 지목된 j.hkjang 을 매핑하고 다시 돌리자 계정 3→2명, 페이지 12→8개, 초안 0→3건으로 움직였습니다.

기록 경로를 따로 만들었습니다. `recordConfluenceError` 는 입력을 `safeConfluenceError` 에 통과시킵니다(v0.68에서 원문 노출을 막으려고 그렇게 했습니다). 그 함수는 알아보지 못한 문자열을 "Confluence에 연결하지 못했습니다"로 바꿉니다. 즉 이 진단을 그 경로로 보내면 화면에서 사라집니다.

`recordConfluenceNotice` 를 따로 두고, 그 이유를 테스트로 고정했습니다 — 나중에 누가 둘을 합치려 할 때 왜 갈라져 있는지 읽을 수 있도록.

원칙으로 남깁니다. "누락"을 세는 지표가 있다고 해서 누락이 보이는 것은 아닙니다. 어느 쪽 집합을 세고 있는지가 전부입니다. 우리 쪽 명부를 세면 상대 쪽에서 들어온 것이 사라지는 것은 영원히 보이지 않습니다.

#### 3.10.25 v0.71.0에서 완료 — 미제출자 253명 중 실제로 늦은 사람은 0명이었습니다

같은 질문을 세 번째로, 이번엔 주간보고 제출 현황에 물었습니다. 이 지표는 **방향이 맞았습니다** — `users` 를 훑으며 보고서가 없는 주를 셉니다. 세는 집합이 옳습니다. 그런데 다른 곳이 틀렸습니다.

303명·15,600건이 든 규모 시험 데이터에서 실측한 "미제출 상위" 목록입니다.

|            |        |                   |            |
|------------|--------|-------------------|------------|
| 부하 기준      | 누락 12주 | 마지막 제출 없음         |            |
| 신입 사원      | 누락 12주 | 마지막 제출 없음         | ← 3일 전 입사  |
| scaleadmin | 누락 12주 | 마지막 제출 없음         | ← 부트스트랩 계정 |
| 사용자 1      | 누락 1주  | 마지막 제출 2026-08-17 |            |
| 사용자 10     | 누락 1주  | 마지막 제출 2026-08-17 |            |
| ...        |        |                   |            |

미제출자 총계 **253명**을 뜯어 보면 250명은 "이번 주(08-24) 미제출"입니다. 마감은 08-31입니다. **아무도 늦지 않았습니다**. 나머지 3명은 3일 전 입사자와 서비스 계정입니다. 관리자가 조치할 목록이 100% 소음이고, 하필 오탐이 정렬 상단을 차지합니다.

두 가지가 겹쳐 있었습니다. **마감 전인 주를 누락으로 셉니다** — 같은 함수의 추이 질의는 마감 규칙을 제대로 쓰는데 미제출 질의만 쓰지 않았습니다. **없던 사람에게 그 주를 묻습니다** — 존재하지 않던 주가 전부 누락으로 잡힙니다.

첫 수정이 틀렸고 실측이 잡았습니다. `u.created_at < 주차 시작일` 로 막았더니 미제출자가 0명이 됐습니다. 시드 사용자가 전부 1~3일 전에 만들어졌기 때문인데, 이건 시드만의 문제가 아닙니다. **과거 보고서를 Import한 실제 배포에서 똑같이 일어납니다** — 전원의 `created_at` 이 오픈 날짜이므로 모든 과거 주차가 그보다 앞서고, 지표가 영원히 0을 보고합니다. 이 제품에는 과거 PPTX를 적재하는 Import 파이프라인이 통째로 있으므로 가정이 아니라 기본 시나리오입니다.

고친 규칙은 **계정 생성일과 첫 제출 주차 중 이른 쪽**입니다. 이관된 사용자는 첫 제출 주차부터, 진짜 신입은 계정 생성일부터 답합니다. 어느 쪽도 지어낸 임계값이 아니라 "그때 있었다"는 증거입니다.

|              | v0.70.0        | v0.71.0     |
|--------------|----------------|-------------|
| 미제출자 총계      | 253명           | 2명          |
| 그중 실제로 늦은 사람 | 0명             | 2명          |
| 목록 1위        | 3일 전 입사자       | 3주 누락자      |
| 진행 중인 주      | 완료된 주와 나란히 16% | "마감 전"으로 분리 |

두 질의를 한 문자열로 묶었습니다. 목록과 총계가 각자 조건을 갖고 있으면 한쪽에 나오고 다른 쪽에 없는 사람이 생깁니다. 그건 두 숫자 중 어느 하나가 틀린 것보다 나쁩니다.

덤으로 마감 문구의 애매함을 없앴습니다. "7일째 되는 날 자정까지"는 그날이 시작하는 자정과 끝나는 자정 두 가지로 읽히고 둘은 하루 차이입니다. 규칙은 그대로 두고 문장만 어느 쪽인지 말하게 했습니다.

원칙으로 남깁니다. **옳은 집합을 세는 것으로는 부족하고, 각자에게 언제부터 물을 수 있는지도 알아야 합니다.** 그리고 그 "언제부터"를 계정 생성일로 잡으면, 과거를 이관한 배포에서는 지표 전체가 조용히 0이 됩니다.

#### 3.10.26 v0.72.0에서 완료 — 최신 2000행 안에서 매긴 순위를 전체 순위라고 불렀습니다  
각도를 바꿔, 규모에서 답이 달라지는 곳을 찾았습니다. 검색이 그랬습니다.

15,594개 보고서·109,175개 항목이 든 데이터에 **제목이 정확히 질의어인 항목 1건**을 1년 전 주차에 심고, 같은 문구를 최근 주차 본문 2,600건에 넣었습니다.

```
v0.71.0 결과 30건 제목 일치 포함 0건
2026-08-24 점수 20 [currentResult]
2026-08-24 점수 20 [currentResult] ← 점수 50짜리는 어디에도 없음
```

원인은 스캔이 `ORDER BY week_start DESC LIMIT 2000` 이라는 것입니다. 순위는 그 2000행 안에서만 매겨지는데, 화면은 그것을 전체에 대한 순위로 보여줍니다. `truncated: true` 가 뭔가 잘렸다고는 말하지만, 남은 것이 최선이라는 주장은 그대로입니다.

첫 시도는 **옳았지만 비쌌습니다.** 정렬을 SQL로 옮겨 `ORDER BY <점수> DESC` 로 바꾸니 제목 일치가 1위로 올라왔습니다. 대신 흔한 단어에서 0.04s → 0.29s, **7배**가 됐습니다. 109,175행 전부에 ILIKE 6개를 평가해야 거의 전부가 같은 점수라는 것을 알게 되기 때문입니다.

대신 값싼 우선 패스를 앞에 뒀습니다. 문단보다 높은 점수를 받는 필드 — 제목과 구분 — 만 따로 묻습니다. 매칭되는 행이 적으므로 싸고, 끝나면 높은 점수는 이미 손에 있습니다.

| 질의              | v0.71.0 | v0.72.0   |
|-----------------|---------|-----------|
| 진행(109,175행 매칭) | 0.031s  | 0.085s    |
| 구축              | 0.054s  | 0.128s    |
| 프로젝트            | 0.184s  | 0.189s    |
| 제목 일치 노출        | 안 됨     | 1위(점수 50) |

우선 패스 예산을 줄여도 빨라지지 않습니다. 500 → 150으로 낮춰 재 봤는데 그대로였습니다. 날짜순 정렬  
이므로 최신 것을 고르려면 이력 전체의 제목 일치를 먼저 다 찾아야 하고, 그 비용은 상한과 무관합니다. 즉  
늘어난 ~60ms는 낭비가 아니라 "전부를 본다"의 값이고, 옛 코드가 빠르면서 틀렸던 이유가 정확히 그것을  
안 봤기 때문입니다. 주석에 그 사실을 적어 뒀습니다 — 다음 사람이 상한을 낮춰 최적화하려다 헛수고하지  
않도록.

인덱스가 하나 빠져 있었습니다. 다른 검색 대상 컬럼은 모두 트라이그램 인덱스가 있는데 `category` 만 없었  
습니다. 4글자 질의에서 47ms → 0.5ms, **94배**입니다(마이그레이션 020). 2글자 한글 질의는 여전히 느리  
고, 이걸 트라이그램이 3글자 미만에서 선택도를 못 갖는 구조적 한계라 인덱스를 더 넣어도 해결되지 않습니  
다 — 확인하고 적어 둡니다.

가드가 처음엔 물지 못했습니다. 테스트가 핸들러가 아니라 테스트 자신이 쓴 SQL을 검사하고 있어서, 우선  
패스를 통째로 지워도 통과했습니다. 스캔을 `searchScan` 으로 뽑아내 테스트가 실제 코드를 부르게 고쳤고,  
이제 패스를 지우거나 대상 필드를 바꾸면 실패합니다. v0.64·v0.68에 이어 세 번째로 같은 실수를 했으니,  
가드를 쓴 뒤 되돌려 보는 것은 선택이 아니라 절차입니다.

원칙으로 남깁니다. 부분집합 위에서 매긴 순위를 전체 순위로 제시하지 않습니다. 잘렸다고 표시하는 것과,  
남은 것이 최선이라고 말하는 것은 다릅니다.

#### 3.10.27 v0.73.0에서 완료 — 짓지 못하는 감시견을 기억이 아니라 절차로 잡습니다

세 릴리즈 연속으로 같은 실수를 했습니다. 특정 결함을 잡으라고 쓴 테스트가 통과했는데, 애초에 실패할 수  
없는 것이었습니다.

- v0.64 — 도달성 가드의 일치 조건이 느슨해 CSS 클래스 이름이 조건을 만족시켰습니다. 유일한 호출부  
를 지워도 통과했습니다.

- v0.68 — 메시지 테스트의 모든 사례가 앞쪽 분기에 걸려, 정작 그 테스트가 존재하는 이유인 폴백이 한 번도 실행되지 않았습니다.
- v0.72 — 순위 테스트가 제품 함수 대신 테스트 자신이 쓴 SQL을 검사했습니다. 그 함수를 통째로 지워도 통과했습니다.

뒤의 둘은 기계적으로 같은 신호를 냅니다. **테스트가 지키겠다는 코드를 실행하지 않았다.** 그건 퍼-테스트 커버리지가 재는 것이고, 따라서 기억해서 확인할 일이 아닙니다.

`scripts/guard-check.py` 를 만들었습니다. 가드 테스트가 자기 대상을 선언합니다.

```
// guards: searchScan, appendSearchMatches
func TestATitleMatchOutranksAFloodOfRecentBodies(t *testing.T) {
```

각 테스트를 혼자 커버리지와 함께 돌려, 선언한 함수에 닿지 않으면 보고하고 종료 코드 1을 냅니다. 검사기 자신도 되돌려 확인했습니다 — v0.72의 실수를 재현하니 `go test` 는 `ok` 인데 검사기가 잡습니다.

**함수에 닿는 것과 정작 그 분기에 닿는 것은 다릅니다.** v0.68의 경우가 그랬습니다. 그래서 임계값을 선언할 수 있게 했습니다.

```
// guards: safeConfluenceError=100
```

"원인마다 다른 문장"이 주장의 전부인 가드에는 이걸 씁니다. 안 닿은 분기가 곧 아무도 확인하지 않은 사례이기 때문입니다.

**27개 가드에 마커를 붙였고, 여섯 개의 진짜 구멍이 나왔습니다.**

| 발견                                                     | 내용                                                                                                                        |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>TestOnlyGenuineSignOutIsMarkedAsNoSession</code> | <code>authenticate</code> 를 한 번도 부르지 <b>않음</b> . <code>errors.Is</code> 의 동작만 확인. <code>authenticate</code> 가 래핑을 그만뒤도 통과 |
| <code>aiUserMessage</code>                             | 예상 못 한 상태 코드의 폴백 문구 미검증                                                                                                   |
| <code>safeConfluenceError</code>                       | 같은 폴백 + <b>Confluence</b> 타임아웃 문구가 한 번도 읽힌 적 <b>없음</b>                                                                    |
| <code>outlookForDueDate</code>                         | 읽을 수 없는 주차, 최근이 더 느린 경우의 문장, 범위로 모자란 경우의 문장 — 3개 분기                                                                       |

| 발견                               | 내용                                         |
|----------------------------------|--------------------------------------------|
| <code>outlookForPeriodEnd</code> | 기간 미설정, 읽을 수 없는 주차, 범위로 모자란 경우 — 3개 분기     |
| CI                               | <b>PostgreSQL이 없어 통합 테스트가 전부 건너뛰어지고 있음</b> |

첫 번째가 가장 무겁습니다. **v0.66**에서 "DB 장애를 로그아웃으로 보고하던" 버그를 고치며 쓴 가드인데, 그 버그를 다시 넣어도 잡지 못했습니다. `authenticate` 를 실제로 부르도록 고치니 이제 잡습니다.

마지막도 큼니다. CI에 데이터베이스가 없어 마이그레이션·제출 현황·검색 순위 검증이 전부 **누군가의 노트북에서만** 돌고 있었습니다. `pgvector/pgvector:pg16` 서비스를 붙였습니다.

**이 검사기가 하지 않는 일도 적어 둡니다.** 단언이 강한지는 보지 않습니다. 되돌려서 실패하는지 확인하는 것은 여전히 진짜 검사이고, 이건 \*되돌릴 것이 애초에 없던\* 경우를 잡습니다.

원칙으로 남깁니다. **세 번 반복한 실수는 규율의 문제가 아니라 절차의 부재입니다.** 사람에게 기억하라고 요구하는 대신, 기계가 물을 수 있는 형태로 바꿉니다.

**#### 3.10.28 v0.74.0에서 완료** — 아무도 닿지 않는 곳을 물었더니, 가장 위험한 두 함수가 나왔습니다 **v0.73**이 "이 가드가 물 수 있는가"를 기계화했으니, 이번엔 반대 질문을 던졌습니다. **아무 테스트도 닿지 않는 곳은 어디인가.**

전체 스위트를 커버리지와 함께 돌리니 **41.5%, 0%**인 함수가 251개입니다. 그중 196개는 `*App` 메서드 — HTTP 핸들러이고, 이 코드베이스에 HTTP 수준 시험 장치가 없다는 알려진 사실입니다. 남은 55개 순수 함수를 읽었고, 둘이 눈에 띄었습니다.

| 함수                           | 하는 일                      | 커버리지 |
|------------------------------|---------------------------|------|
| <code>workScope.where</code> | 누가 누구의 업무를 볼 수 있는지 정하는 술어 | 0%   |
| <code>safeImportPath</code>  | 경로 탈출을 막는 검사              | 0%   |

둘 다 틀리면 조용히 위험하고, 둘 다 아무도 확인한 적이 없었습니다.

**둘 다 맞았습니다.** 테스트를 써서 확인했고, 그 자체가 결과입니다. `where` 는 역할마다 올바르게 좁히고 비어 있지 않으며, `SelfOnly` 가 관리자까지 이깁니다. `safeImportPath` 는 `..` 탈출, 다른 작업 디렉터리, **번호가 겹치는 작업(5 대 51)**, 확장자 덧붙이기를 모두 거부합니다.

대신 다른 것이 나왔습니다. 술어는 `w` 와 `u` 라는 SQL 별칭에 의존하고, 네 호출부 중 세 곳에 이런 주석이 있습니다.

별칭을 바꾸면 이 코드는 컴파일되고 필터만 조용히 사라집니다.

같은 위험을 세 번 적어 뒀다는 것은 그것이 진짜라는 뜻입니다. 그런데 지키는 것은 사람의 주의력뿐이었습니다. 이제 모든 `scope.where()` 호출부의 질의문이 `w` 와 `u` 를 실제로 묶는지 기계가 확인합니다. 별칭을 바꾸거나, 호출부가 사라지거나, 가드가 볼 대상이 없어지는 세 경우 모두 실패하는 것을 확인했습니다.

Go가 이미 막아 주는 쪽도 적어 뒀습니다 — 술어를 질의에 붙이지 않으면 `where` 가 미사용 변수가 되어 빌드가 깨집니다. 별칭만 미끄러질 수 있습니다.

검사가 제가 방금 쓴 테스트를 잡았습니다. `scopeForPrincipal` 을 지킨다고 선언해 놓고 부르지 않았습다. v0.73에서 만든 절차가 만든 당일에 저를 잡은 셈이고, 이것이 절차가 기억보다 나은 이유입니다.

원칙으로 남깁니다. 주석으로 세 번 적은 위험은 가드로 옮길 때가 된 것입니다.

#### 3.10.29 v0.75.0에서 완료 — 핸들러 196개가 통째로 미검증이던 것

v0.74에서 커버리지 0%인 함수 251개 중 \*\*196개가 `*App` 메서드 — HTTP 핸들러\*\*라는 것을 확인하고 "이 코드베이스에 HTTP 수준 시험 장치가 없다는 알려진 사실"이라고 적었습니다. 알려진 사실은 고쳐도 되는 사실입니다.

이 제품이 반환하는 모든 권한 규칙, 오류 코드, 상태 코드가 그 안에 있습니다. 지금까지 그것을 확인한 방법은 컨테이너를 띄우고 손으로 두드리는 것이었고, 그건 누군가 기억할 때만, 그리고 그 순간 떠올린 경로에 대해서만 일어납니다.

바이너리가 쓰는 것과 같은 mux에 요청을 보내는 장치를 만들었습니다. 실제 `New` 로 앱을 세우므로 마이그레이션·부트스트랩·미들웨어가 전부 진짜입니다.

두 번 돌리자 깨졌고, 그게 설계를 결정했습니다. 처음에는 계정 이름을 고정했다가 `USERNAME_EXISTS` 를 만났습니다. 이름을 실행마다 유일하게 만들었더니 이번엔 전부 `INVALID_CREDENTIALS` 였습니다 — 부트스트랩 관리자는 관리자가 하나도 없을 때만 생성되므로, 이전 실행이 남긴 행 하나가 이후 모든 실행을 막습니다. 즉 스위치가 자기 정리에 의존하고 있었습니다.

정리를 더 잘하는 대신 의존을 없앴습니다. 테스트마다 자기 데이터베이스를 만들고 끝나면 지웁니다. 두 번 연속 돌려 통과하고 임시 DB가 남지 않는 것을 확인했습니다.

첫 네 개의 테스트로 확인한 것 — 전부 읽었을 때는 맞았고, 그것을 강제하는 경로로는 한 번도 지나간 적이 없던 규칙들입니다.



- 남의 보고서는 읽을 수도, 고칠 수도, 지울 수도 없습니다
- `requireRole` 은 계층이 아니라 허용 목록이므로 USER는 작은 ADMIN이 아닙니다
- 쿠키 없는 요청은 403이 아니라 401이고, 오류 코드를 함께 냅니다
- 다른 출처에서 온 쓰기는 유효한 세션을 갖고 있어도 `CSRF_REJECTED`

|             | v0.74.0 | v0.75.0 |
|-------------|---------|---------|
| 구문 커버리지     | 41.5%   | 45.6%   |
| 커버리지 0%인 함수 | 251개    | 214개    |
| 가드          | 30개     | 34개     |

네 개의 테스트가 37개 함수를 새로 덮었습니다 — 인증, 로그인, 로그인 제한, CSRF 미들웨어 전체가 여기 들어 있습니다. 값은 이 네 테스트가 아니라 장치에 있습니다. 지금까지 이 반복에서 찾은 결함 상당수가 이것 이 먼저 있었다면 애초에 잡혔을 것입니다.

원칙으로 남깁니다. "그건 구조적으로 못 한다"는 말은 대개 "아직 안 했다"입니다. 두 번은 다르게 취급해야 합니다.

#### 3.10.30 v0.76.0에서 완료 — 검토 워크플로 전체가 한 번도 지나간 적 없었습니다

v0.75의 장치를 처음으로 제대로 써 봤습니다. 대상은 검토 워크플로 — 제출·승인·반려·재수정입니다. 이 제품이 "주간보고 플랫폼"인 이유의 절반이 여기 있고, 커버리지는 0%였습니다.

전부 옳았습니다. 다섯 가지 규칙을 핸들러로 확인했고 모두 맞습니다.

| 규칙                                       | 결과                              |
|------------------------------------------|---------------------------------|
| 자기 보고서는 자기가 승인 못 함 — 관리자도                | 403                             |
| USER는 누구 것도 검토 못 함                       | 403                             |
| 팀장은 자기 조직만, 다른 조직은 못 함                   | 200 / 403                       |
| 검토는 <code>SUBMITTED</code> 에서만, 두 번은 안 됨 | 409 <code>INVALID_STATUS</code> |
| 반려에는 사유가 필요하고, 없으면 상태도 안 움직임             | 400                             |

가장 확인하고 싶었던 것은 승인된 보고서를 작성자가 고칠 수 있는가였습니다. 고칠 수 있고, 고치면 `DRAFT`로 되돌아가며 `reviewed_by`가 지워지고 이력에 `"작성자가 제출·승인 후 내용을 수정하여 재검토 상태로 전환"`이 남습니다. 승인이 내용보다 오래 살지 않습니다.

읽어서 맞는 것과 강제 경로로 확인한 것은 다르므로, 세 가지를 되돌려 전부 실패하는 것을 봤습니다 — 본인 검토 차단 제거, `AND status='SUBMITTED'` 제거, 승인 뒤 수정 시 상태 유지. 마지막 변이에서는 이력 행동 함께 사라져 두 단언이 동시에 터집니다.

검사가 또 저를 잡았습니다. `canReviewReport=100`을 걸었더니 50%였습니다. 빠진 절반이 팀장이 자기 조직원을 검토하는 정상 경로 — 이 제품에서 가장 흔한 검토 — 였습니다. 조직을 만들어 채웠습니다.

나머지 절반은 `p.Role == "USER"` 분기인데, 승인 라우트가 이미 `requireRole`로 `USER`를 막으므로 요청으로는 닿을 수 없는 방어선입니다. 100%를 요구하면 그 함수를 직접 부르는 테스트를 쓰게 되고, 그건 라우트가 아니라 함수를 시험하는 것입니다. 임계값을 내리고 그 이유를 주석으로 적었습니다 — 다음 사람이 "왜 여기만 100%가 아니지"를 다시 묻지 않도록.

|             | v0.75.0 | v0.76.0 |
|-------------|---------|---------|
| 구문 커버리지     | 45.6%   | 46.7%   |
| 커버리지 0%인 함수 | 214개    | 207개    |
| 가드          | 34개     | 39개     |

원칙으로 남깁니다. 모든 규칙에 **100%**를 요구하면 규칙이 아니라 함수를 시험하게 됩니다. 요구를 낮출 때는 낮춘 이유를 남깁니다.

#### 3.10.31 v0.77.0에서 완료 — 이름을 언급하기만 하면 통과하던 가드

Import 확정 경로를 보러 갔다가 다른 것을 찾았습니다.

확정은 `WHERE user_id=$1`로 자기 보고서만 건드립니다. 남의 주간보고를 교체하거나 병합할 수 없습니다. 여기까지는 안전한데, 눈에 걸린 것이 하나 있었습니다 — `REPLACE`가 승인된 보고서의 내용을 통째로 바꾸면서 `reviewed_by`는 지우지 않습니다. v0.76에서 확인한 "승인이 내용보다 오래 살지 않는다"와 어긋납니다.

그래서 `reviewed_by`가 화면에 나오는지 봤습니다. 어디에도 안 나옵니다. 그런데 이 프로젝트에는 v0.65부터 `"응답에 실려 나가지만 화면이 읽지 않는 필드"`를 잡는 가드가 있습니다. 왜 못 잡았는지 보니, 가드가 이렇게 되어 있었습니다.

```
if !strings.Contains(frontend, field) { ... }
```

프론트엔드 어딘가에 그 이름이 들어 있으면 통과합니다. `types.ts` 의 타입 선언만으로 충분합니다. v0.64 에서 CSS 클래스 이름이 도달성 조건을 만족시킨 것과 같은 모양입니다.

세어 봤습니다. 응답 필드 335개 중 **26개**가 `types.ts` 에만 있고 화면 코드 어디에서도 쓰이지 않습니다.

그런데 화면이 안 쓰는 것이 곧 죽은 것은 아닙니다. 이 제품은 개인 API 키와 MCP 서버로도 읽힙니다. 26개 중 7개는 `docs/openapi.yaml` 산문에 설명이 있었습니다 — 화면 대신 연동하는 쪽이 소비합니다. 즉 옳은 기준은 "화면이 읽는가"가 아니라 "화면이 그리거나 문서가 적는가"입니다.

그 기준으로 다시 세니 **19개가 어디에도 없습니다**. 계산해서 網선으로 내보내는데, 그리지도 적지도 않습니다. 연동하는 사람은 존재 자체를 알 방법이 없습니다.

| 무리             | 필드                                                                                                                                                                                                 |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 수명 주기 시각       | <code>submittedAt</code> <code>reviewedAt</code> <code>startedAt</code> <code>completedAt</code> <code>analyzedAt</code> <code>confirmedAt</code><br><code>builtAt</code> <code>generatedAt</code> |
| 기간 보고 인사<br>이트 | <code>continuingItems</code> <code>oneOffItems</code> <code>noLandingItems</code> <code>sourceReports</code> <code>sourceItems</code><br><code>dedupRate</code>                                    |
| 그 밖            | <code>periodEnd</code> <code>overlapWeeks</code> <code>latestIssue</code> <code>scannedCharacters</code> <code>roles</code>                                                                        |

쓰레기가 아니라 문서화되지 않은 **API 표면**입니다. 전부 `docs/openapi.yaml` 에 뜻과 함께 적었고, 가드는 `types.ts` 를 빼고 보되 문서를 함께 보도록 고쳤습니다.

되돌려서 확인했습니다. 문서에서 인사이트 여섯 줄을 지우면 새 가드가 실패합니다. 그리고 옛 가드로 되돌리면 문서를 통째로 지워도 통과합니다 — 19개를 애초에 잡을 수 없었다는 증거입니다.

원칙으로 남깁니다. "어딘가에 그 이름이 있다"는 사용의 증거가 아닙니다. 그리고 소비자가 화면 하나뿐이라고 가정하면, API를 가진 제품에서는 살아 있는 것을 죽었다고 부르게 됩니다.

#### 3.10.32 v0.78.0에서 완료 — 바뀐 본문에 남아 있던 승인 도장

v0.77에서 `reviewedAt` 을 문서에 적으면서 이렇게 썼습니다.

검토자가 승인 또는 반려한 시각. 작성자가 내용을 고치면 다시 `null` 이 됩니다.

문서에 적고 나니 확인해야 할 것이 생겼습니다. PPTX Import로 내용을 바꾸는 것도 작성자가 내용을 고치는 것입니다. 그때도 그런가?

아니었습니다.

```
승인된 보고서를 PPTX로 통째로 교체한 뒤
summary      = "PPTX에서 가져온 내용"  ← 본문은 완전히 바뀜
reviewedAt    = 2026-08-24 17:02:30    ← 그대로
reviewed_by   = 1                      ← 그대로
```

이전 문장을 승인한 사람이, 자기가 본 적 없는 문장의 검토자로 남습니다. 편집기로 고치면( `updateReport` ) 둘 다 지워집니다. Import 경로만 빠져 있었습니다. 병합도 같습니다 — 아무도 검토하지 않은 문단을 승인된 보고서에 더하면서 도장은 유지했습니다.

두 곳 다 `reviewed_at=NULL, reviewed_by=NULL` 을 넣었고, 각각 독립적으로 되돌려 두 테스트가 따로 실패하는 것을 확인했습니다.

같은 종류가 더 있는지 전수로 봤습니다. `UPDATE weekly_reports SET` 은 여섯 곳입니다.

| 위치                                  | 하는 일                    | 검토 도장                                                                     |
|-------------------------------------|-------------------------|---------------------------------------------------------------------------|
| <code>reports.go</code> 제출          | DRAFT → SUBMITTED       | 지움 ✓                                                                      |
| <code>reports.go</code> 검토          | SUBMITTED → APPROVED/반려 | 찍음 ✓                                                                      |
| <code>reports.go</code> 수정          | 본문 변경                   | 지움 ✓                                                                      |
| <code>imports.go</code> REPLACE     | 본문 교체                   | 안 지움 → 고침                                                                 |
| <code>imports.go</code> MERGE       | 본문 추가                   | 안 지움 → 고침                                                                 |
| <code>confluence_handlers.go</code> | 자동 초안 수락                | <code>DRAFT / REVISION_REQUESTED</code> 에서만 동작하고 본문은 편집기 저장으로 들어옴 — 해당 없음 |

문서를 적은 것이 결함을 찾아냈습니다. v0.77에서 19개 필드에 뜻을 붙인 것은 문서화 작업이었는데, 그중 한 줄이 검증 가능한 약속이 되면서 코드가 그 약속을 어기고 있다는 것이 드러났습니다.

원칙으로 남깁니다. 행동을 문장으로 적으면 그 문장이 시험이 됩니다. 적을 수 없다면 아직 정하지 않은 것이고, 적었는데 다르다면 둘 중 하나가 틀린 것입니다.

#### 3.10.33 v0.79.0에서 완료 — 한 문장, 두 가지 구현

v0.78이 통했던 이유 — 문서에 적은 행동이 시험이 된다 — 를 넓혀 봤습니다. docs/openapi.yaml 에서 행동으로 서술된 약속을 추려 실제로 그런지 확인했습니다.

가장 여러 엔드포인트에 걸친 약속은 상한 규칙입니다.

limit 기본값 200, 최대 500. 범위를 벗어나면 잘라 맞추고, 읽을 수 없는 값은 기본값으로 본다.

네 개 목록 엔드포인트에 여섯 가지 입력(생략·상한 초과·0·음수·비숫자·상한 자체)을 넣어 봤습니다.

저부터 틀렸습니다. limit=0 이 기본값을 줄 것이라 기대했는데 1을 줍니다. 다시 읽으니 계약은 "잘라 맞추고"이므로 1이 맞습니다 — 제 기대가 틀렸습니다. 그런데 그 실수 자체가 신호였습니다. 방금 그 문장을 다시 읽은 사람도 헛갈렸다면, 연동하는 사람은 더 헛갈립니다. 문장을 고쳤습니다.

범위를 벗어나면 가까운 끝으로 잘라 맞추다 — 상한보다 크면 상한, 1보다 작으면 1이다(0이나 음수도 기본값이 아니라 1이다).

그리고 진짜가 하나 나왔습니다. /admin/audit 만 다르게 답합니다.

|                          | limit=0 | limit=-5 |
|--------------------------|---------|----------|
| clampQueryInt — 5개 엔드포인트 | 1       | 1        |
| auditLogs — 직접 구현        | 50      | 50       |

같은 규칙을 문서에 한 번 적어 놓고 구현이 둘이었습니다. /reports 에서 규칙을 배운 클라이언트가 /admin/audit 에서 다른 답을 받습니다. offset 도 마찬가지로 혼자 다른 형태였습니다. 둘 다 공용 헬퍼로 모았습니다.

되돌려 확인했습니다 — audit을 직접 구현으로 되돌리면 실패하고, 상한을 무시하게 하면 실패하고, 읽을 수 없는 값의 기본값 처리를 없애면 실패합니다. 음수 offset 이 500이 되지 않는지도 함께 고정했습니다 (PostgreSQL은 음수 OFFSET에 오류를 냅니다).

원칙으로 남깁니다. 규칙을 문장으로 한 번 적었다면 구현도 한 곳이어야 합니다. 문장은 하나인데 구현이 둘이면, 둘 중 하나는 문서를 읽은 사람을 배신합니다.

#### 3.10.34 v0.80.0에서 완료 — 문서의 숫자와 코드의 숫자를 묶었습니다

v0.79에서 상한 규칙 하나를 확인했으니, 이번엔 docs/openapi.yaml 이 숫자로 못 박은 약속 전부를 훑었습니다. 열두 개가 나왔습니다 — 목록 상한, 검색 단어 수, 병목 20건, 관련 업무 5건, 다이제스트 10건, 유사·중복 상위 200건, 결정 50건, 검색 단계 전환 5건.

전부 맞았습니다. 열두 개 모두 코드의 상수와 일치합니다.

그러면 이 반복의 값은 무엇인가. 맞다는 것을 확인한 것 자체도 결과지만, 더 중요한 것은 이 일치가 유지된다는 보장이 하나도 없었다는 사실입니다. 누군가 병목 상한을 25로 올리면 문서는 20이라고 계속 말합니다. 오프라인망에서 연동하는 사람에게 이 파일은 유일한 근거이고, 자신 있게 틀린 문서는 아무 말도 안 하는 것보다 나쁩니다.

그래서 기댓값을 상수에서 만들어 냅니다.

```
 {"병목", fmt.Sprintf("최대 %d건 담는다", bottleneckLimit)},
```

상수를 바꾸면 찾는 문장이 바뀌고, 문서가 따라오지 않으면 실패합니다. 세 개를 실제로 바꿔 전부 실패하는 것을 확인했습니다.

매직 넘버 셋을 상수로 뽑았습니다. 문서가 말하는데 코드에는 이름이 없던 숫자들입니다 — Import 상한 200, 검색 단계 전환 기준 5. 이름 없는 숫자는 이 검사가 볼 수 없고, 볼 수 없으면 지킬 수도 없습니다.

원칙으로 남깁니다. 문서가 말하는 숫자에는 코드에 이름이 있어야 합니다. 그래야 둘이 어긋나는 순간을 기계가 압니다.

#### 3.10.35 v0.81.0에서 완료 — 아무것도 켜지 않은 상태, 그리고 묻지 못한 제 테스트

이 제품이 실제로 설치되는 모습은 모든 선택 기능이 꺼진 상태입니다. AI 게이트웨이 없음, Confluence 없음, 검토 워크플로 없음. 문서는 그 상태에서 무엇을 답하기로 했는지 적어 두었지만, 그 약속을 확인한 적은 없었습니다.

전부 옳았습니다.

| 상황                               | 답                       |
|----------------------------------|-------------------------|
| AI 꺼짐 · 본문 분석 / 결정 제안 / PPTX 업로드 | 503 AI_UNAVAILABLE      |
| 워크플로 꺼짐 · 승인                     | 409 WORKFLOW_DISABLED   |
| Confluence 꺼짐 · 강제 동기화           | 409 CONFLUENCE_DISABLED |

| 상황               | 답      |
|------------------|--------|
| 아무것도 안 켜 채 읽기 7종 | 전부 200 |

500이 아니라 503·409라는 것이 요점입니다. 500은 "이 제품이 고장 났다"로 읽히고, 빈 200은 "찾을 게 없었다"로 읽힙니다. 둘 다 누군가 바꿀 수 있는 설정을 감춥니다.

확장이 하나도 없는 PostgreSQL도 흉내 냈습니다. `word_similarity` 와 `<=>` 를 쓰는 세 함수가 모두 자기 능력 플래그 뒤에 있는지 읽고, 실제로 꺼서 확인했습니다. (읽는 중에 `semanticWorkItems` 가 `pg_trgm` 함수를 `pgvector` 플래그로 막는 것처럼 보였는데, `<=>` 여서 맞았습니다.)

그리고 이번 반복의 진짜 수확은 여기서 나왔습니다. 제가 방금 쓴 테스트 세 개가 물지 못했습니다.

- 확장 테스트는 `존재하지않는말` 로 검색했습니다. 아무것도 매치되지 않으니 뒤 단계도 빈손이고, 게이트를 통째로 제거해도 `fuzzy` 는 `false` 로 남습니다.
- AI 테스트는 게이트웨이를 **아예 설정하지 않은** 상태로 돌렸습니다. 활성 여부 검사를 느슨하게 해도 주소가 없어 어차피 실패하니 티가 나지 않습니다.

`guard-check.py` 는 이걸 못 잡습니다 — 두 테스트 모두 대상 함수를 실행하긴 했기 때문입니다. v0.73에서 "단언이 강한지는 보지 않는다"고 적어 둔 한계가 그대로 나타났고, 되돌려 보는 절차가 여전히 진짜 검사인 이유입니다.

고쳤습니다. 검색은 매치되는 오타( `확장업는` )를 쓰고, 확장이 켜진 상태에서 `fuzzy` 가 `true` 인 것을 먼저 확인한 뒤 끕니다. AI는 설정은 되어 있고 꺼져만 있는 상태로 만듭니다. 이제 둘 다 되돌리면 실패합니다.

원칙으로 남깁니다. 없는 것을 확인하는 테스트는 있는 것으로 먼저 확인해야 합니다. 꺼진 상태만 보면, 애초에 켜도 안 되는 것과 구분되지 않습니다.

#### 3.10.36 v0.82.0에서 완료 — 틀린 비밀번호가 통과해도 아무도 몰랐습니다

v0.73이 `"테스트가 대상을 실행했는가"`를 기계화했고, v0.81이 그것으로 부족하다는 것을 보여 줬습니다. 실행하고도 물지 못하는 테스트는 커버리지로 구분되지 않습니다. 코드를 실제로 망가뜨려 보는 것으로만 구분됩니다.

`scripts/mutation-check.py` 를 만들었습니다. `// guards: F` 마커가 이미 짝을 알고 있으니, F 안에서만 비교 연산자를 뒤집고 `&& ↔ ||` 를 바꾸고 반환 상수를 돌린 뒤 그 테스트를 돌립니다.

첫 실행에서 도구 자체의 결함이 나왔습니다. 살아남은 변이를 그대로 보고했더니, `canReviewReport` 의 어떤 변이는 자기 가드는 놓치지만 다른 테스트가 잡습니다. 둘을 섞으면 멀쩡한 곳을 구멍이라 부르게 됩니다. 살아남을 때마다 전체 스위트를 한 번 더 돌려 둘로 나누게 했습니다.



| 건수                          |                     |
|-----------------------------|---------------------|
| 적용한 변이                      | 204 (컴파일 불가 11건 제외) |
| 지목된 가드가 잡음                  | 97                  |
| 다른 테스트가 잡음 — 마커가 엉뚱한 곳을 가리킴 | 43                  |
| 아무도 못 잡음                    | 64                  |

64건 중 둘이 즉시 눈에 띄었고, 둘 다 인증입니다.

`login` 의 `||` 를 `&&` 로 바꾸면 틀린 비밀번호로 로그인됩니다. 전체 스위트가 통과합니다. 이 제품에서 가장 중요한 단언 — \*틀린 비밀번호는 통과하지 못한다\* — 이 한 번도 검증된 적이 없었습니다. 기존 로그인 테스트는 전부 올바른 비밀번호로 들어가거나 저장소 장애를 다뤘습니다.

**API 키 SQL**의 `expires_at>now()` 를 `<` 로 뒤집으면 만료된 키가 통과하고 살아 있는 키가 거부됩니다. 개인 API 키 경로에 테스트가 하나도 없었습니다 — 이 제품이 MCP 서버와 연동을 노출하는 바로 그 경로입니다.

둘 다 테스트를 썼고, v0.81의 교훈대로 **긍정 대조**를 먼저 둡니다: 올바른 비밀번호가 통과하는 것, 갓 발급한 키가 동작하는 것을 확인한 뒤에 거부를 확인합니다. 덤으로 폐기된 키와 발급된 적 없는 토큰도 막습니다. 없는 계정과 틀린 비밀번호가 같은 오류 코드인 것도 고정했습니다 — 다르면 그 차이가 어떤 아이디가 실재하는지 알려 줍니다.

세 변이를 되돌려 전부 실패하는 것을 확인했습니다.

남은 **62건**은 목록으로 둡니다. 상당수는 의미상 무해할 것이고, 판별에는 사람의 판단이 필요합니다.

`python3 -u scripts/mutation-check.py` 가 언제든지 다시 만들어 냅니다. 도구는 CI 단계가 아닙니다 — 한 번 도는 데 40분 넘게 걸리고, `scale-check.py` 처럼 필요할 때 돌리는 물건입니다.

원칙으로 남깁니다. 테스트가 통과한다는 것은 코드가 옳다는 뜻이 아니라, 이 테스트가 이 코드를 틀리게 만들 방법을 모른다는 뜻입니다.

#### 3.10.37 v0.83.0에서 완료 — 남은 62건을 훑었더니, 제 주석이 틀렸습니다

v0.82의 변이 검사가 남긴 "아무도 못 잡는 변이 62건"을 훑었습니다. 함수별로 묶으니 `uploadImportPPTX` 4건, `deleteReport` 4건, `csrf` 1건이 눈에 띄었습니다.

업로드 상한 넷이 전부 뒤집혀도 아무도 몰랐습니다. 파일 개수, 파일 크기, 빈 파일, 요청 전체 크기 — 로그인한 누구나 파일을 보낼 수 있는 엔드포인트의 방어선입니다. 개수 3개로 제한해 놓고 4개를 보내면 400이

고, 1MB 제한에 1MB+1을 보내면 그 파일만 실패로 기록됩니다. 거부하면서 기록을 남기는지까지 확인했습니다 — 기록 없는 거부는 올린 사람이 알 방법이 없습니다.

삭제의 버전 검사도 전부 미검증이었습니다. 두 사람이 같은 이름으로 서로 다른 것을 지우지 못하게 하는 장치인데, 버전 없음·0·음수·비숫자·낡은 버전 다섯 경우 모두 통과했습니다. 다섯 번의 거부 뒤에도 보고서가 그대로 있는 것, 그리고 **맞는 버전으로는 실제로 지워지는 것**(공정 대조)까지 확인했습니다.

세 번째에서 제가 틀렸습니다. `csrf` 의 `p.AuthType == "session"` 조건을 넓히는 변이가 살아남았고, v0.75에 제가 이렇게 적어 뒀습니다.

API 키는 브라우저가 아니고 출처를 갖지 않으므로 이 검사가 적용되면 안 된다 — 그러지 않으면 모든 연동이 깨진다.

확인해 보니 **개인 API 키는 조회 전용**입니다. 쓰기가 아예 없으니 깨질 연동도 없습니다. 그 조건은 `canReviewReport` 의 USER 분기와 같은 **닿을 수 없는 방어선**이고, 제 주석은 과장이었습니다. 더 나쁜 것은 그 주석이 \*검사한다고 약속하면서 검사하지 않았다\*는 점입니다 — 읽는 사람은 그 규칙이 지켜지고 있다고 믿게 됩니다.

주석을 사실로 고치고, **실제로 성립하는 규칙**을 대신 고정했습니다: 조회 전용 키는 어떤 쓰기도 하지 못하며, 거부 코드가 `API_KEY_SCOPE_DENIED` 여서 연동하는 쪽이 이유를 압니다. 그 검사를 제거하는 변이를 넣으니 **조회 전용 키가 보고서를 만들었습니다**.

원칙으로 남깁니다. 주석이 약속한 검사는 코드가 하고 있어야 합니다. 하지 않는다면 그 주석은 설명이 아니라 잘못된 보증입니다.

#### 3.10.38 v0.84.0에서 완료 — 정시와 시각이 같은 칸이어도 아무도 몰랐습니다

남은 변이를 계속 훑었습니다. 이번에 걸린 것은 v0.71이 절반만 끝낸 일입니다.

v0.71에서 마감 시각을 Go와 SQL이 똑같이 계산하는지는 고정했습니다. 그런데 그 시각이 무엇을 위한 것인지 — 마감 전에 낸 보고서가 정시로, 뒤에 낸 것이 시각으로 세어지는지 — 는 확인한 적이 없습니다.

```
count(*) FILTER (WHERE r.submitted_at <  deadline.at)    -- 정시
count(*) FILTER (WHERE r.submitted_at >=  deadline.at)    -- 시각
```

한 줄만 뒤집어도 전체 스위트가 통과합니다. 뒤집힌 상태의 출력이 그 무의미함을 보여 줍니다 — 보고서 하나가 `onTime=1 late=1` 로 **양쪽에 동시에** 세어집니다. 관리자 화면의 "기한 내 제출"은 그 두 숫자로 만들어 집니다.

마감 한 시간 전·한 시간 후·정확히 마감 시각 세 경우를 고정했습니다. 경계에서는 어느 쪽이 가져가든 **합이 1** 이어야 한다는 것만 요구합니다 — 규칙이 어느 쪽인지보다, 한 보고서가 두 번 세어지지 않는 것이 먼저입니다.

테스트를 쓰다 제가 그 버그를 그대로 재현했습니다. 마감 시각을 `time.UTC` 로 계산했더니 실제보다 아홉 시간 늦어져, 마감 한 시간 전에 낸 보고서가 지각으로 나왔습니다. `v0.71`이 고친 바로 그 실수 — 서비스 시간 대가 아닌 곳에서 순간을 만드는 것 — 를 테스트 안에서 다시 저지른 셈이고, 주석에 그렇게 적어 뒀습니다.

업무 검색의 두 글자 하한도 고정했습니다. 한 글자 질의는 오류가 아니라 **빈 결과**입니다 — 화면이 타이핑할 때마다 요청을 보내기 때문입니다. 다만 검색이 \*실행되어서는\* 안 됩니다. 비교를 뒤집으면 진짜 질의가 전부 빈손이 되고 한 글자가 전수를 훑는데, 아무도 몰랐습니다. 처음엔 400을 기대했다가 실제 설계가 더 낫다는 것을 확인하고 기대를 고쳤습니다.

원칙으로 남깁니다. 계산이 맞는지 고정했다면, 그 계산이 무엇을 결정하는지도 고정해야 합니다. 시각이 정확한 것과 그 시각으로 옳게 나누는 것은 다른 두 가지입니다.

#### 3.10.39 `v0.85.0`에서 완료 — 위험 목록에 완료된 업무가 올라와도 아무도 몰랐습니다

남은 번이 중 예측 판정 계열 넷을 봤습니다. 전부 **경계와 제외 규칙**이고, 전부 관리자 화면에 틀린 문장을 내놓는 부류입니다.

`noLandingDate` 의 제외 조건을 `||` 에서 `&&` 로 바꾸면 완료된 업무가 "끝날 주차를 못 잡는 업무" 목록에 **오릅니다**. 정제 업무를 빼는 규칙은 테스트가 있었는데, 바로 옆의 완료 업무 제외는 없었습니다. 주석이 그 이유를 이미 적어 두고 있었습니다 — \*"같은 업무를 두 제목으로 부르는 것이 위험 목록이 안 읽히게 되는 방식"\*. 완료된 업무를 위험이라 부르는 것은 그보다 나쁩니다.

마감일이 마지막 보고 주차에 걸리면 `weeksLeft` 가 **0**입니다. `<= 0` 을 `< 0` 으로 바꾸면 그 경우가 추정 분기로 넘어가, 이미 도착한 날짜에 대해 예보를 내놓습니다. 기간 보고 쪽도 같은 모양이었습니다.

넷 다 고정했고, 각각 되돌려 따로 실패하는 것을 확인했습니다. 긍정 대조를 먼저 둔 것도 같습니다 — `noLandingDate` 가 \*참\*을 내야 하는 두 경우를 먼저 놓지 않으면, 어디서나 거짓을 내는 구현도 통과합니다.

원칙으로 남깁니다. 규칙에 제외가 있다면 제외마다 사례가 있어야 합니다. 하나를 시험하고 그 옆을 비워 두면, 빠진 쪽이 정확히 그 규칙이 막으려던 것을 통과시킵니다.

### 3.11 `v0.21` — Source-to-Report Agent

- 커밋·결재·Jira에서 사용자 동의 아래 근거를 수집해 `WorkItem` 후보와 출처 링크를 생성합니다. `v0.17`의 `provenance` 모델을 전제합니다.

## 3.12 v1.0 — Enterprise Work Intelligence Platform

- 조직 Knowledge Graph. 사람·업무·시스템·프로젝트·문서 관계를 질의합니다.
- 의미 기반 검색 자체는 v0.12에서 기반이 놓였습니다(4.4). 남은 과제는 검색 결과를 관계 질의로 확장하는 것입니다.

## 4. 선행 기술 결정 사항

로드맵 후반에서 발견하면 비용이 큰 결정입니다. 해당 단계 이전에 확정합니다.

### 4.1 ReportItem 저장 방식 변경 (v0.11에서 해결)

보고서 수정은 `report_items` 를 전량 삭제하고 다시 삽입해 저장할 때마다 행 식별자가 바뀌었습니다. 그 상태에서는 `work_item_id` 를 추가해도 다음 저장에서 연결이 사라집니다.

v0.11에서 1번(안정 식별자 기준 재조정)을 적용했습니다. 제출된 항목 중 기존 행을 가리키는 것은 갱신하고, 새 항목만 삽입하며, 사라진 항목만 삭제합니다. 세 번 연속 저장해도 행 식별자와 업무 연결이 유지되는 것을 확인했습니다.

### 4.2 상태 볼륨에 저장하는 데이터

비밀 설정 키(4.1의 사례)와 첨부 이미지 파일이 `/var/lib/weekly` 에 있고, 나머지는 PostgreSQL에 있습니다. 두 저장소가 분리돼 있으므로 **한쪽만 유지한 업그레이드는 조용히 깨집니다**. 행은 남고 파일만 사라지므로 목록은 정상으로 보이고 조회만 404가 됩니다.

v0.14.2에서 기동 시 유실을 감지해 로그로 남기고, 목록 응답이 파일 존재 여부를 반환하도록 했습니다. 이후 상태 볼륨에 파일을 추가로 저장하는 기능은 **같은 방식의 무결성 확인을 함께 제공해야 합니다**.

v0.15.0에서 감지에 예방을 더했습니다. 백업 도구가 두 저장소를 같은 복구 지점으로 묶고, 참조된 첨부 파일 수와 실제 보관된 파일 수를 비교해 부족하면 0이 아닌 종료 코드를 냅니다. 순서도 임의가 아닙니다. 업로드가 파일을 먼저 쓰고 행을 나중에 넣으므로 **DB를 먼저 덤프하고 파일을 나중에 복사해야** 덤프에 있는 행의 파일이 반드시 존재합니다.

### 4.3 자동 판정의 위치

제목 유사도 병합, Decision 추출, 변화 분류는 모두 **제안**이어야 하며 사용자 확정 없이 확정 데이터로 승격하지 않습니다. v0.6.0의 병합 오판 사례(한 글자 차이 업무가 하나로 합쳐짐)가 근거입니다.

v0.18.0에서 마지막 예외를 없앴습니다. 기간 보고가 저장된 식별자를 무시하고 제목으로 다시 추측하고 있었고, 그 결과 사용자가 방금 나눈 업무를 한 화면이 도로 합칠 수 있었습니다. **자동 판정을 쓰는 곳은 저장된 정정이 있으면 그것을 먼저 봐야 합니다.**

v0.11의 WorkItem은 이 원칙을 지키지 못한 채 출발했습니다. 제목 정규화 결과가 곧 확정되었고 정정 경로가 없었습니다. v0.16.0의 병합·분리로 해소했습니다. 이후 자동 판정을 저장 데이터로 쓰는 기능은 **정정 경로를 같은 릴리즈에 함께 제공해야 하며, 그 정정이 다음 자동 판정에도 살아남아야 합니다.**

#### 4.4 임베딩 저장 방식 (v0.12에서 해결)

의미 기반 검색은 임베딩이 필요합니다. 오프라인망에서 `pgvector` 설치를 전제할 수 없어 세 가지 안을 두고 2번(배열 컬럼 + 애플리케이션 코사인 계산)을 권장했습니다.

v0.12에서 **1번( `pgvector` 선택 설치 )** 을 채택했습니다. 권장을 바꾼 근거는 다음과 같습니다.

- 실제 운영 대상 PostgreSQL에 `pg_trgm` 과 `pgvector` 가 이미 설치돼 있음이 확인됐습니다. 전제할 수 없다는 판단의 근거가 사라졌습니다.
- 확장 부재가 장애가 되지 않도록 마이그레이션이 확장 생성 실패를 흡수하고, 기동 시 감지한 능력에 따라 해당 검색 단계만 비활성화합니다. 즉 1번을 택해도 3번의 안전성을 유지합니다.
- 2번은 같은 안전성을 얻는 대신 정렬과 상한 처리를 애플리케이션으로 가져와야 하고, 확장이 있는 환경에서도 그 비용을 계속 냅니다.

임베딩 생성은 AI Gateway를 사용하므로 AI 비활성 환경에서는 의미 검색이 비활성화되고 앞의 두 단계만 동작합니다.

#### 4.5 유사도 판정에 쓸 함수와 임계값

측정 없이 고르면 안 되는 항목입니다. v0.12에서 실제 말뭉치로 측정해 결정했습니다.

- 검색에는 `word_similarity` : `similarity` 는 질의어가 문서 전체에서 차지하는 비중을 재므로 긴 제목 속의 짧은 검색어를 놓칩니다( `전표검증` → `월결산 자동화` 0.214). `word_similarity` 는 0.400으로 찾아 냅니다.
- 워드 클라우드 합산에는 유사도를 쓰지 않습니다: 합쳐야 할 `전표 검증` · `전표검증` (0.375)이 합치면 안 되는 `서버 A 점검` · `서버 B 점검` (0.600)보다 낮아 임계값이 존재하지 않습니다. 공백 제거 후 정확 일치로 판정합니다.
- 의미 검색 임계값은 고정하지 않습니다: 코사인 점수 범위가 임베딩 모델마다 다릅니다. 설정으로 노출하고 관리자 화면에 임베딩 현황을 함께 제공합니다.

## 5. 리소스 및 품질 관리 전략

- **테스트 자동화:** Go 유닛 테스트, PostgreSQL 격리 통합 테스트, React 타입 검사와 Production Build를 릴리즈마다 실행합니다.
- **무중단 마이그레이션:** 스키마는 전진 전용으로 적용하고 기존 데이터는 변경하지 않습니다. 자동 down migration은 제공하지 않습니다.
- **오프라인 우선:** 모든 신규 기능은 AI Gateway와 외부 연동이 없는 환경에서도 핵심 동작이 성립해야 합니다. AI는 품질을 높이는 선택 계층으로 둡니다.
- **근거 보존:** 자동 생성 결과는 출처를 함께 저장하고, 근거를 확인할 수 없는 결과는 저장 후보로 사용하지 않습니다.
- **회귀 검증:** 릴리즈마다 실제 컨테이너를 기동해 주요 흐름을 확인하고 릴리즈 본문에 검증 내역을 남깁니다.